



Siemens
Industry
Online
Support

TIA Scripting Python

Using Python scripts to automate TIA Portal processes.

SIEMENS

Legal information

Use of application examples

Application examples illustrate the solution of automation tasks through an interaction of several components in the form of text, graphics and/or software modules. The application examples are a free service by Siemens AG and/or a subsidiary of Siemens AG ("Siemens"). They are non-binding and make no claim to completeness or functionality regarding configuration and equipment. The application examples merely offer help with typical tasks; they do not constitute customer-specific solutions. **The application examples are not subject to standard tests and quality inspections of a chargeable product and may contain functional and performance defects or other faults and security vulnerabilities. You are responsible for the proper and safe operation of the products in accordance with all applicable regulations, including checking and customizing the application example for your system, and ensuring that only trained personnel use it in a way that prevents property damage or injury to persons. You are solely responsible for any productive use.**

Siemens grants you the non-exclusive, non-sublicensable and non-transferable right to have the application examples used by technically trained personnel. Any change to the application examples is your responsibility. Sharing the application examples with third parties or copying the application examples or excerpts thereof is permitted only in combination with your own products. Any further use of the application examples is explicitly not permitted and further rights are not granted. You are not allowed to use application examples in any other way, including, without limitation, for any direct or indirect training or enhancements of AI models.

Disclaimer of liability

Siemens shall not assume any liability, for any legal reason whatsoever, including, without limitation, liability for the usability, availability, completeness and freedom from defects of the application examples as well as for related information, configuration and performance data and any damage caused thereby. This shall not apply in cases of mandatory liability, for example under the German Product Liability Act, or in cases of intent, gross negligence, or culpable loss of life, bodily injury or damage to health, non-compliance with a guarantee, fraudulent non-disclosure of a defect, or culpable breach of material contractual obligations. Claims for damages arising from a breach of material contractual obligations shall however be limited to the foreseeable damage typical of the type of agreement, unless liability arises from intent or gross negligence or is based on loss of life, bodily injury or damage to health. The foregoing provisions do not imply any change in the burden of proof to your detriment. You shall indemnify Siemens against existing or future claims of third parties in this connection except where Siemens is mandatorily liable.

By using the application examples you acknowledge that Siemens cannot be held liable for any damage beyond the liability provisions described.

Other information

Siemens reserves the right to make changes to the application examples at any time without notice and to terminate your use of the application examples at any time. In case of discrepancies between the suggestions in the application examples and other Siemens publications such as catalogs, the content of the other documentation shall have precedence.

The Siemens terms of use (<https://www.siemens.com/global/en/general/terms-of-use>) shall also apply.

Cybersecurity information

Siemens provides products and solutions with industrial cybersecurity functions that support the secure operation of plants, systems, machines and networks.

In order to protect plants, systems, machines and networks against cyber threats, it is necessary to implement – and continuously maintain – a holistic, state-of-the-art industrial cybersecurity concept. Siemens' products and solutions constitute one element of such a concept.

Customers are responsible for preventing unauthorized access to their plants, systems, machines and networks. Such systems, machines and components should only be connected to an enterprise network or the internet if and to the extent such a connection is necessary and only when appropriate security measures (e.g. firewalls and/or network segmentation) are in place.

For additional information on industrial cybersecurity measures that may be implemented, please visit www.siemens.com/cybersecurity-industry.

Siemens' products and solutions undergo continuous development to make them more secure. Siemens strongly recommends that product updates are applied as soon as they are available and that the latest product versions are used. Use of product versions that are no longer supported, and failure to apply the latest updates may increase customer's exposure to cyber threats.

To stay informed about product updates, subscribe to the Siemens Industrial Cybersecurity RSS Feed under <https://www.siemens.com/cert>.

Table of contents

1.	TIA Scripting Python	17
1.1.	TIA Portal programming interface.....	17
1.2.	Prerequisites	17
1.2.1.	Installed software	17
1.3.	Python environment.....	17
1.3.1.	Python installation.....	17
1.3.2.	Python Quickstart	18
1.4.	Managing rights of TIA Portal Openness	19
1.4.1.	Add user to Siemens TIA Openness group	19
1.4.2.	Openness security-dialog	19
1.5.	How to use TIA Scripting Python.....	20
1.5.1.	Use TIA Scripting Python via file import	20
1.5.2.	Use TIA Scripting Python as Python package	20
1.6.	Scripts.....	20
1.7.	Create an Executable from your script	22
1.7.1.	Steps to create an executable.....	22
1.8.	Hierarchy	24
2.	Function List.....	25
2.1.	Devices - Hmi	26
2.1.1.	get_name()	26
2.1.2.	get_hmi_type()	26
2.1.3.	open_device_editor().....	26
2.1.4.	compile_hardware()	26
2.1.5.	compile_software()	26
2.1.6.	upgrade_hardware(full_upgrade: bool)	26
2.1.7.	get_property(name: str).....	27
2.1.8.	get_properties()	27
2.1.9.	set_property(name:str, value: str)	27
2.1.10.	get_identifier()	27
2.1.11.	get_hmi_tag_tables(folder_path: Optional[str] = None)	27
2.1.12.	get_screens(folder_path: Optional[str] = None)	28
2.1.13.	get_text_lists()	28
2.1.14.	get_scripts(folder_path: Optional[str] = None).....	28
2.1.15.	get_alarms().....	28
2.1.16.	get_alarm_classes()	28
2.1.17.	get_connections()	28
2.1.18.	get_cycles().....	28

2.1.19.	get_graphic_lists()	29
2.1.20.	get_global_screen_elements()	29
2.1.21.	get_screen_overview()	29
2.1.22.	get_slide_in_screens(folder_path: Optional[str] = None)	29
2.1.23.	import_hmi_tags(import_root_directory: str, target_folder_path: Optional[str] = None)	29
2.1.24.	import_connections(import_root_directory: str)	29
2.1.25.	import_cycles(import_root_directory: str)	29
2.1.26.	import_scripts(import_root_directory: str, target_folder_path: Optional[str] = None)	30
2.1.27.	import_text_lists(import_root_directory: str)	30
2.1.28.	import_graphic_lists(import_root_directory: str)	30
2.1.29.	import_screens(import_root_directory: str, target_folder_path: Optional[str] = None)	30
2.1.30.	import_popup_screens(import_root_directory: str, target_folder_path: Optional[str] = None)	30
2.1.31.	import_template_screens(import_root_directory: str, target_folder_path: Optional[str] = None)	31
2.1.32.	import_slidein_screens(import_root_directory: str)	31
2.1.33.	import_global_elements(import_file: str)	31
2.1.34.	import_screen_overview(import_file: str)	31
2.2.	Devices - Plc	32
2.2.1.	get_name()	32
2.2.2.	open_device_editor()	32
2.2.3.	get_online_state()	32
2.2.4.	go_offline()	32
2.2.5.	go_online(pc_interface_type: str, pc_interface_name: str, target_interface: str)	32
2.2.6.	download(pc_interface_type: str, pc_interface_name: str, target_interface: str)	32
2.2.7.	download_to_memory_card(download_directory: str, is_plcsim_advanced: Optional[bool] = None)	33
2.2.8.	get_plc_tag_tables(folder_path: Optional[str] = None)	33
2.2.9.	update_module_description()	33
2.2.10.	compile_hardware()	33
2.2.11.	compile_software()	33
2.2.12.	upgrade_hardware(full_upgrade: bool)	33
2.2.13.	compare_to_online()	34
2.2.14.	get_program_blocks(folder_path: Optional[str] = None)	34
2.2.15.	get_system_blocks()	34
2.2.16.	get_user_data_types(folder_path: Optional[str] = None)	34
2.2.17.	get_external_sources(folder_path: Optional[str] = None)	34
2.2.18.	get_force_tables()	34
2.2.19.	get_watch_tables(folder_path: Optional[str] = None)	35
2.2.20.	get_technology_objects(folder_path: Optional[str] = None)	35
2.2.21.	get_software_units()	35
2.2.22.	get_safety_administration()	35
2.2.23.	import_blocks(import_root_directory: str, target_folder_path: Optional[str] = None)	35

2.2.24.	import_plc_tags(import_root_directory: str, target_folder_path: Optional[str] = None)	35
2.2.25.	import_data_types(import_root_directory: str, target_folder_path: Optional[str] = None)	36
2.2.26.	import_technology_objects(import_root_directory: str, target_folder_path: Optional[str] = None)	36
2.2.27.	import_watch_tables(import_root_directory: str, target_folder_path: Optional[str] = None)	36
2.2.28.	create_software_unit(name: str)	36
2.2.29.	safety_print(print_file: str)	37
2.2.30.	get_property(name: str)	37
2.2.31.	get_properties()	37
2.2.32.	set_property(name:str, value: str)	37
2.2.33.	get_identifier()	37
2.3.	Enums	39
2.3.1.	PortalMode	39
2.3.2.	UmacUserMode	39
2.3.3.	ExportFormats	39
2.3.4.	ExportOptions	39
2.3.5.	CleanUpMode	39
2.3.6.	LibraryExportOptions	39
2.3.7.	HarmonizeOptions	39
2.3.8.	DependenciesMode	39
2.4.	Global - Global	40
2.4.1.	open_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)	40
2.4.2.	attach_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)	40
2.4.3.	open_attach_project(project_file_path: str, portal_mode: Optional[Enums.PortalMode] = None, server_project_view: Optional[bool] = None)	40
2.4.4.	get_installed_bundles()	40
2.4.5.	get_installed_products()	41
2.4.6.	set_umac_credentials(user_name: str, user_password: str, user_type: Enums.UmacUserMode)	41
2.4.7.	encrypt_umac_config(umac_file_path: str, secret: str, secret_env_name: str)	41
2.4.8.	set_logging(path: str, console: bool)	41
2.5.	Global - Portal	42
2.5.1.	get_process_id()	42
2.5.2.	open_project(project_file_path: str, server_project_view: Optional[bool] = None)	42
2.5.3.	open_project_with_copy(project_file_path: str, target_directory_path: str, delete_existing_project: bool)	42
2.5.4.	retrieve_archive(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)	42
2.5.5.	create_project(target_directory_path: str, project_name: str, delete_existing_project: bool)	43
2.5.6.	get_project()	43
2.5.7.	get_global_library_infos()	43
2.5.8.	get_global_library(library_name: str)	43
2.5.9.	open_global_library(library_path: str)	43
2.5.10.	open_global_library_with_copy(target_directory_path: str, library_path: str, delete_existing_project: bool)	43

2.5.11.	<code>create_global_library(target_directory_path: str, library_name: str, delete_existing_project: bool)</code>	44
2.5.12.	<code>retrieve_archive_library(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)</code>	44
2.5.13.	<code>close_portal()</code>	44
2.5.14.	<code>show_online_finger_prints(mode: str, pc_interface: str, ip_address: str)</code>	44
2.5.15.	<code>get_project_servers(server: Optional[str] = None)</code>	45
2.5.16.	<code>detach()</code>	45
2.5.17.	<code>add_project_server(url: str, name: str)</code>	45
2.6.	Global - Product	46
2.6.1.	<code>get_name()</code>	46
2.6.2.	<code>get_release()</code>	46
2.6.3.	<code>get_version()</code>	46
2.7.	Global - ProductBundle	47
2.7.1.	<code>get_title()</code>	47
2.7.2.	<code>get_release()</code>	47
2.7.3.	<code>get_products()</code>	47
2.8.	HMI Data - HmiAlarm	48
2.8.1.	<code>get_name()</code>	48
2.8.2.	<code>get_property(name: str)</code>	48
2.8.3.	<code>get_properties()</code>	48
2.8.4.	<code>set_property(name:str, value: str)</code>	48
2.8.5.	<code>get_identifier()</code>	48
2.9.	HMI Data - HmiAlarmClass	49
2.9.1.	<code>get_name()</code>	49
2.9.2.	<code>get_property(name: str)</code>	49
2.9.3.	<code>get_properties()</code>	49
2.9.4.	<code>set_property(name:str, value: str)</code>	49
2.9.5.	<code>get_identifier()</code>	49
2.10.	HMI Data - HmiConnection	50
2.10.1.	<code>get_name()</code>	50
2.10.2.	<code>get_property(name: str)</code>	50
2.10.3.	<code>get_properties()</code>	50
2.10.4.	<code>set_property(name:str, value: str)</code>	50
2.10.5.	<code>export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)</code>	50
2.10.6.	<code>get_identifier()</code>	51
2.11.	HMI Data - HmiCycle	52
2.11.1.	<code>get_name()</code>	52
2.11.2.	<code>get_property(name: str)</code>	52
2.11.3.	<code>get_properties()</code>	52
2.11.4.	<code>set_property(name:str, value: str)</code>	52

2.11.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	52
2.11.6.	get_identifier()	53
2.12.	HMI Data - HmiGraphicList	54
2.12.1.	get_name()	54
2.12.2.	get_property(name: str).....	54
2.12.3.	get_properties()	54
2.12.4.	set_property(name:str, value: str)	54
2.12.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	54
2.12.6.	get_identifier()	55
2.13.	HMI Data - HmiScreen.....	56
2.13.1.	get_name()	56
2.13.2.	get_property(name: str).....	56
2.13.3.	get_properties()	56
2.13.4.	set_property(name:str, value: str)	56
2.13.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	56
2.13.6.	get_identifier()	57
2.14.	HMI Data - HmiScript	58
2.14.1.	get_name()	58
2.14.2.	get_property(name: str).....	58
2.14.3.	get_properties()	58
2.14.4.	set_property(name:str, value: str)	58
2.14.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	58
2.14.6.	get_identifier()	59
2.15.	HMI Data - HmiTag	60
2.15.1.	get_name()	60
2.15.2.	get_property(name: str).....	60
2.15.3.	get_properties()	60
2.15.4.	set_property(name:str, value: str)	60
2.15.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	60
2.15.6.	get_identifier()	61
2.16.	HMI Data - HmiTagTable	62
2.16.1.	get_name()	62
2.16.2.	get_property(name: str).....	62
2.16.3.	get_properties()	62
2.16.4.	set_property(name:str, value: str)	62

2.16.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	62
2.16.6.	get_identifier()	63
2.17.	HMI Data - HmiTextList	64
2.17.1.	get_name()	64
2.17.2.	get_property(name: str).....	64
2.17.3.	get_properties()	64
2.17.4.	set_property(name:str, value: str)	64
2.17.5.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	64
2.17.6.	get_identifier()	65
2.18.	Library - GlobalLibrary.....	66
2.18.1.	get_name()	66
2.18.2.	save()	66
2.18.3.	get_author()	66
2.18.4.	get_path().....	66
2.18.5.	is_modified()	66
2.18.6.	is_read_only()	66
2.18.7.	update_library(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None, library_name: Optional[str] = None)	66
2.18.8.	update_project(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None)	67
2.18.9.	harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)	67
2.18.10.	archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)	68
2.18.11.	get_property(name: str).....	68
2.18.12.	get_properties()	68
2.18.13.	set_property(name:str, value: str)	68
2.18.14.	clean_up(folder_path: Optional[str] = None).....	69
2.18.15.	get_types().....	69
2.18.16.	find_library_type(library_type_name: str)	69
2.18.17.	close_global_library()	69
2.18.18.	create_folder(folder_path: str)	69
2.18.19.	delete_folder(folder_path: str)	69
2.19.	Library - GlobalLibraryInfo	70
2.19.1.	get_name()	70
2.19.2.	get_property(name: str).....	70
2.19.3.	get_properties()	70
2.19.4.	set_property(name:str, value: str)	70
2.20.	Library - LibraryType	71
2.20.1.	get_name()	71
2.20.2.	get_author()	71

2.20.3.	get_guid()	71
2.20.4.	get_versions()	71
2.20.5.	find_version(version: str)	71
2.20.6.	get_property(name: str)	71
2.20.7.	get_properties()	71
2.20.8.	set_property(name:str, value: str)	72
2.20.9.	get_path_full()	72
2.20.10.	get_path()	72
2.20.11.	get_comment()	72
2.20.12.	get_identifier()	72
2.21.	Library - LibraryTypeFolder	73
2.21.1.	get_name()	73
2.21.2.	get_folders()	73
2.21.3.	get_types()	73
2.21.4.	find_library_type(library_type_name: str)	73
2.21.5.	find_folder(folder_name: str)	73
2.22.	Library - LibraryTypeVersion	74
2.22.1.	get_author()	74
2.22.2.	get_guid()	74
2.22.3.	get_version_number()	74
2.22.4.	get_modified_date()	74
2.22.5.	get_state()	74
2.22.6.	get_type_object()	74
2.22.7.	get_property(name: str)	74
2.22.8.	get_properties()	75
2.22.9.	set_property(name:str, value: str)	75
2.22.10.	find_instances(device_name: Optional[str] = None)	75
2.22.11.	instantiate(device_name: str, software_unit_name: Optional[str] = None, folder_path: Optional[str] = None)	75
2.22.12.	get_supported_export_format()	76
2.22.13.	export(target_directory_path: str, export_format: str, library_export_options: Enums.LibraryExportOptions, keep_folder_structure: Optional[bool] = None)	76
2.22.14.	edit(device_name: Optional[str] = None)	76
2.22.15.	release(dependencies_mode: Enums.DependenciesMode, version: str, author: str, comment: str)	76
2.22.16.	discard()	76
2.22.17.	set_as_default()	77
2.22.18.	get_comment()	77
2.22.19.	get_identifier()	77
2.23.	Library - ProjectLibrary	78
2.23.1.	get_types()	78
2.23.2.	find_library_type(library_type_name: str)	78

2.23.3.	clean_up(clean_up_mode: Enums.CleanUpMode, folder_path: Optional[str] = None)	78
2.23.4.	import_library_types(import_root_directory: str, device_name: Optional[str] = None, software_unit_name: Optional[str] = None, plc_folder_path: Optional[str] = None, library_folder_path: Optional[str] = None)	78
2.23.5.	update_library(library_name: str, update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None)	79
2.23.6.	update_project(delete_mode: int, type_guids: Optional[List[str]] = None)	79
2.23.7.	harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)	79
2.23.8.	create_folder(folder_path: str)	80
2.23.9.	delete_folder(folder_path: str)	80
2.24.	PLC Data - ExternalSource	81
2.24.1.	get_name()	81
2.24.2.	get_property(name: str)	81
2.24.3.	block_gen()	81
2.24.4.	delete()	81
2.24.5.	get_properties()	81
2.24.6.	set_property(name:str, value: str)	81
2.24.7.	get_supported_export_format()	82
2.24.8.	get_identifier()	82
2.25.	PLC Data - ForceTable	83
2.25.1.	get_name()	83
2.25.2.	get_property(name: str)	83
2.25.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	83
2.25.4.	is_consistent()	83
2.25.5.	show_in_editor()	83
2.25.6.	get_properties()	84
2.25.7.	set_property(name:str, value: str)	84
2.25.8.	get_supported_export_format()	84
2.25.9.	get_path_full()	84
2.25.10.	get_path()	84
2.25.11.	get_fingerprints()	84
2.25.12.	get_identifier()	85
2.26.	PLC Data - NamedValueType	86
2.26.1.	get_name()	86
2.26.2.	get_namespace()	86
2.26.3.	export(target_directory_path: str, keep_folder_structure: Optional[bool] = None)	86
2.27.	PLC Data - PlcTag	87
2.27.1.	get_name()	87
2.27.2.	get_property(name: str)	87
2.27.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	87

2.27.4.	delete()	87
2.27.5.	get_properties()	87
2.27.6.	set_property(name:str, value: str)	88
2.27.7.	get_supported_export_format()	88
2.27.8.	get_identifier()	88
2.28.	PLC Data - PlcTagTable	89
2.28.1.	get_name()	89
2.28.2.	get_property(name: str)	89
2.28.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	89
2.28.4.	get_plc_tags()	89
2.28.5.	get_user_constants()	89
2.28.6.	export_cross_references(target_directory_path: str, filter: int)	90
2.28.7.	show_in_editor()	90
2.28.8.	delete()	90
2.28.9.	get_properties()	90
2.28.10.	set_property(name:str, value: str)	90
2.28.11.	get_supported_export_format()	90
2.28.12.	get_path_full()	91
2.28.13.	get_path()	91
2.28.14.	get_fingerprints()	91
2.28.15.	get_identifier()	91
2.29.	PLC Data - ProgramBlock	92
2.29.1.	get_name()	92
2.29.2.	get_property(name: str)	92
2.29.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	92
2.29.4.	compile()	92
2.29.5.	is_consistent()	92
2.29.6.	export_cross_references(target_directory_path: str, filter: int)	93
2.29.7.	show_in_editor()	93
2.29.8.	get_type_version_guid()	93
2.29.9.	get_type_guid()	93
2.29.10.	delete()	93
2.29.11.	get_properties()	93
2.29.12.	get_path_full()	93
2.29.13.	get_path()	93
2.29.14.	set_property(name:str, value: str)	94
2.29.15.	get_supported_export_format()	94
2.29.16.	get_fingerprints()	94

2.29.17.	is_library_type()	94
2.29.18.	set_know_how_protection(password: str)	94
2.29.19.	remove_know_how_protection(password: str)	94
2.29.20.	get_identifier()	95
2.30.	PLC Data - SafetyAdministration	96
2.30.1.	is_logged_on()	96
2.30.2.	is_password_set()	96
2.30.3.	get_offline_serial_number()	96
2.30.4.	export_config(target_directory_path: str)	96
2.30.5.	import_config(import_root_directory: str)	96
2.31.	PLC Data - SoftwareUnit	97
2.31.1.	get_name()	97
2.31.2.	compile()	97
2.31.3.	export_configuration(export_file_path: str)	97
2.31.4.	import_configuration(import_file_path: str)	97
2.31.5.	get_plc_tag_tables()	97
2.31.6.	get_program_blocks()	97
2.31.7.	get_system_blocks()	97
2.31.8.	get_user_data_types()	98
2.31.9.	get_external_sources()	98
2.31.10.	get_named_value_types()	98
2.31.11.	import_blocks(import_root_directory: str)	98
2.31.12.	import_plc_tags(import_root_directory: str)	98
2.31.13.	import_data_types(import_root_directory: str)	98
2.31.14.	export_cross_references(target_directory_path: str, filter: int)	98
2.31.15.	get_property(name: str)	99
2.31.16.	get_properties()	99
2.31.17.	delete()	99
2.31.18.	set_property(name:str, value: str)	99
2.31.19.	get_identifier()	99
2.32.	PLC Data - SystemBlock	100
2.32.1.	get_name()	100
2.32.2.	get_property(name: str)	100
2.32.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	100
2.32.4.	compile()	100
2.32.5.	is_consistent()	100
2.32.6.	export_cross_references(target_directory_path: str, filter: int)	101
2.32.7.	show_in_editor()	101
2.32.8.	delete()	101

2.32.9.	get_properties()	101
2.32.10.	get_path_full()	101
2.32.11.	get_path()	101
2.32.12.	set_property(name:str, value: str)	101
2.32.13.	get_supported_export_format()	102
2.32.14.	get_fingerprints()	102
2.32.15.	get_identifier()	102
2.33.	PLC Data - TechnologyObject	103
2.33.1.	get_name()	103
2.33.2.	get_property(name: str)	103
2.33.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	103
2.33.4.	compile()	103
2.33.5.	is_consistent()	103
2.33.6.	delete()	104
2.33.7.	get_properties()	104
2.33.8.	get_path_full()	104
2.33.9.	get_path()	104
2.33.10.	set_property(name:str, value: str)	104
2.33.11.	get_supported_export_format()	104
2.33.12.	get_fingerprints()	104
2.33.13.	get_identifier()	105
2.34.	PLC Data - UserConstant	106
2.34.1.	get_name()	106
2.34.2.	get_property(name: str)	106
2.34.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	106
2.34.4.	delete()	106
2.34.5.	get_properties()	106
2.34.6.	set_property(name:str, value: str)	107
2.34.7.	get_supported_export_format()	107
2.34.8.	get_identifier()	107
2.35.	PLC Data - UserData Type	108
2.35.1.	get_name()	108
2.35.2.	get_property(name: str)	108
2.35.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	108
2.35.4.	compile()	108
2.35.5.	is_consistent()	108
2.35.6.	export_cross_references(target_directory_path: str, filter: int)	109
2.35.7.	get_type_version_guid()	109

2.35.8.	get_type_guid().....	109
2.35.9.	delete().....	109
2.35.10.	get_properties()	109
2.35.11.	get_path_full()	109
2.35.12.	get_path().....	109
2.35.13.	set_property(name:str, value: str)	110
2.35.14.	get_supported_export_format()	110
2.35.15.	get_fingerprints()	110
2.35.16.	is_library_type()	110
2.35.17.	get_identifier()	110
2.36.	PLC Data - WatchTable	111
2.36.1.	get_name()	111
2.36.2.	get_property(name: str).....	111
2.36.3.	export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)	111
2.36.4.	is_consistent().....	111
2.36.5.	show_in_editor()	111
2.36.6.	delete().....	112
2.36.7.	get_properties()	112
2.36.8.	set_property(name:str, value: str)	112
2.36.9.	get_supported_export_format()	112
2.36.10.	get_path_full()	112
2.36.11.	get_path().....	112
2.36.12.	get_fingerprints()	112
2.36.13.	get_identifier()	113
2.37.	Project - Project	114
2.37.1.	get_portal().....	114
2.37.2.	save()	114
2.37.3.	close()	114
2.37.4.	set_simulation_support(value: bool)	114
2.37.5.	archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)	114
2.37.6.	save_as(target_directory_path: str, project_name: str)	114
2.37.7.	export_cax_data(export_file_path: str, log_file_path: str)	115
2.37.8.	import_cax_data(import_file_path: str, log_file_path: str)	115
2.37.9.	open_topology_editor()	115
2.37.10.	open_network_editor().....	115
2.37.11.	get_plcs().....	115
2.37.12.	get_hmis()	115
2.37.13.	get_application_tests()	116
2.37.14.	get_system_tests()	116

2.37.15.	get_rule_sets()	116
2.37.16.	get_project_library()	116
2.37.17.	web_block_generate()	116
2.37.18.	upgrade_hardware(full_upgrade: bool)	116
2.37.19.	sivarc_generate()	117
2.37.20.	update_module_description()	117
2.37.21.	set_virtual_plc_support(value: bool)	117
2.37.22.	import_umac_config(import_file_path: str)	117
2.37.23.	export_umac_config(export_file_path: str)	117
2.37.24.	import_password_policy(import_file_path: str)	117
2.37.25.	export_password_policy(export_file_path: str)	118
2.37.26.	delete()	118
2.37.27.	start_transaction(undo_text: str, dialog_text: str)	118
2.37.28.	end_transaction(rollback: Optional[bool] = None)	118
2.37.29.	update_transaction(dialog_text: str)	118
2.37.30.	get_property(name: str)	118
2.37.31.	get_properties()	119
2.37.32.	set_property(name:str, value: str)	119
2.37.33.	is_session()	119
2.37.34.	commit_and_close(commit_message: str)	119
2.37.35.	get_identifier()	119
2.38.	Project - ProjectServer	120
2.38.1.	get_host()	120
2.38.2.	get_port()	120
2.38.3.	get_server_name()	120
2.38.4.	print_info()	120
2.38.5.	get_property(name: str)	120
2.38.6.	get_properties()	120
2.38.7.	set_property(name:str, value: str)	120
2.38.8.	delete()	121
2.38.9.	add_project(project_path: str)	121
2.38.10.	create_local_session(server_project_path: str, session_directory_path: str, session_name: str, exclusive: Optional[bool] = None)	121
2.38.11.	get_server_project_paths(group: Optional[str] = None)	121
2.38.12.	delete_local_session(session_file_path: str, server_project_path: str)	121
2.39.	Test Suite - ApplicationTest	123
2.39.1.	get_name()	123
2.39.2.	get_property(name: str)	123
2.39.3.	export(target_directory_path: str)	123
2.39.4.	set_scope(plc_name: str, instance_name: Optional[str] = None, execution_mode: Optional[int] = None)	123

2.40.	Test Suite - RuleSet	124
2.40.1.	get_name()	124
2.40.2.	get_property(name: str).....	124
2.40.3.	export(target_directory_path: str)	124
2.41.	Test Suite - SystemTest	125
2.41.1.	get_name()	125
2.41.2.	export(target_directory_path: str)	125
2.41.3.	set_scope(opcua_server_address: str, opcua_server_interface_type: int, opcua_server_interface_folder_path: Optional[str] = None)	125
2.41.4.	get_property(name: str).....	125
2.41.5.	get_properties()	125
2.41.6.	set_property(name:str, value: str)	126
2.41.7.	get_identifier()	126
3.	Appendix	127
3.1.	Service and support	127
3.2.	Links and literature	128
3.3.	Change documentation	128

1. TIA Scripting Python

TIA Scripting Python is a module for writing Python scripts to interact with the TIA Portal Openness interface. It is designed to automate simple tasks in TIA Portal projects using a simple scripting language, and thus avoiding the need for complicated programming. It helps simplify tasks such as importing and compiling program blocks, exporting data, and comparing projects. This module simplifies some of the basic tasks of TIA Portal and makes it easier for users without much experience to work with TIA Portal Openness. In addition, the installed TIA Portal versions are automatically recognized and used.

1.1. TIA Portal programming interface

The TIA Portal Openness API (Application Programming Interface) provided by TIA Portal allows you to automate recurring steps in your projects. This is useful as manual adjustments in projects involve a high susceptibility to errors. Moreover, this automation allows you to save time, enabling you to work more efficiently.

1.2. Prerequisites

1.2.1. Installed software

In order to run this program, the following software has to be installed:

- TIA Portal V15.1 (or newer)
- TIA Portal Openness V15.1 (or newer)

The requirements for the mentioned programs can be found in their respective documentations.

To work with TIA Scripting Python, Python is required.

- Python 3.12.X, 3.13.X, or 3.14.X
- any Python IDE, e.g. Visual Studio Code

1.3. Python environment

1.3.1. Python installation

To check if you have Python installed on a Windows PC, search in the start bar for Python or run the following on the Command Line (cmd.exe):

```
python --version
```

If no Python version is installed, you can download it from <https://www.python.org/>. Python version 3.12.X, 3.13.X, or 3.14.X is required to use.

Note

For pip commands to work, Python must be in your system's PATH environment variable.

1. When installing Python, make sure to check the option "Add python.exe to PATH" in the installer.
2. If Python is already installed and not in your PATH, you can add it manually:
 - Find the folder where `python.exe` is installed (e.g., `C:\Users\YourName\AppData\Local\Programs\Python\Python312\`).
 - Open the Start menu, search for "Environment Variables", and select **Edit the system environment variables**.
 - In the System Properties window, click **Environment Variables...**
 - Under **System variables** (or **User variables**), find and select the **Path** variable, then click **Edit**.
 - Click **New** and add the path to the folder containing `python.exe`.
 - Click **OK** to save and close all dialogs.
3. After updating the PATH, restart any open command prompts or terminals.

NOTICE

TIA Scripting Python requires **Python version 3.12.X, 3.13.X, or 3.14.X**.

You must use Python 3.12, 3.13, or 3.14 (any patch version, e.g., 3.12.0, 3.13.1, 3.14.0, etc.).

Other versions, including older ones like 3.11.X or newer ones beyond 3.14.X, are not supported and will not work.

If you already have a different Python version installed, you still need to install one of the supported versions (3.12.X, 3.13.X, or 3.14.X) specifically.

1.3.2. Python Quickstart

Python is an interpreted programming language, which means that as developers write Python (.py) files in a text editor and then put those files into the Python interpreter to be executed.

The way to run a Python file is like this on the command line:

```
python helloworld.py
```

Where “helloworld.py” is the name of your Python file.

1.4. Managing rights of TIA Portal Openness

1.4.1. Add user to Siemens TIA Openness group

When you install TIA Portal Openness on the PC, the “Siemens TIA Openness” user group is automatically created. Whenever you access the TIA Portal with your TIA Portal Openness application, the TIA Portal verifies that you are a member of the “Siemens TIA Openness” user group, either directly or indirectly by way of another user group. If you are a member of the “Siemens TIA Openness” user group, the TIA Portal Openness application starts and establishes a connection to the TIA Portal.

Follow the instructions within the [Openness manual](#) for detailed instructions.

1.4.2. Openness security-dialog

When you use the application the first time and it connects to TIA Portal via TIA Portal Openness, TIA Portal will prompt you to accept or reject the connection via a security dialog:

- If you just want to connect your TIA Portal Openness application to the TIA Portal once, click “Yes” at the prompt. The next time your TIA Portal Openness application tries to connect the TIA Portal, the prompt will be shown again.
- To create a whitelist entry for your TIA Portal Openness application follow these steps:
 - Click “Yes to all” at the prompt to display an User Account Control Dialog.
 - Click “Yes” at the User Account Control Dialog to add your application to the whitelist in the windows registry and to attach the application to the TIA Portal.

Further information can be found in the [Openness manual](#).

1.5. How to use TIA Scripting Python

There are two possibilities to use TIA Scripting Python.

1.5.1. Use TIA Scripting Python via file import

Unzip TiaScriptingPython_x.x.x.zip to a folder on your hard-drive. To make this folder known during execution of scripts, set a global environment variable "TIA_SCRIPTING", whereas the value is the path of the previously unzipped TIA Scripting Python binaries. This way, "TIA_SCRIPTING" will be used to find the Python module of the TIA Scripting Python by Python scripts.

1.5.2. Use TIA Scripting Python as Python package

You can install TIA Scripting Python as a Python package using `pip` (Python's package manager). Follow these steps:

Installation Steps:

1. Open a command prompt or terminal.
2. Navigate to the directory where the `siemens_tia_scripting-x.x.x-cp31x-cp31x-win_amd64.whl` file is located:

```
cd C:\Path\To\Your\TIA_Scripting_Python\binaries\
```

(Replace `C:\Path\To\Your\TIA_Scripting_Python\binaries\` with the actual path to your TIA Scripting Python binaries folder.)

3. Install the package using `pip`:

```
pip install siemens_tia_scripting-x.x.x-cp31x-cp31x-win_amd64.whl
```

- Replace `x.x.x` with the actual version number of your TIA Scripting Python package (e.g., `1.0.0`).
- The `cp31x-cp31x-win_amd64` part may vary depending on your Python version (e.g.: 3.12, 3.13 and 3.14) and system architecture, so use the exact filename you have.

What happens after installation? This command will install the TIA Scripting Python package into your current Python environment's `site-packages` directory. This makes its modules and functionalities available for import and use in your Python scripts.

Intellisense Support

Intellisense support can be used only if you install TIA Scripting Python as Python package. Pylance language server (Python Intellisense) is able to provide full Intellisense Support. Pylance is default by Visual Studio Code Python Plugin and Microsoft Visual Studio Python. Other Language servers like "Jedi" are currently untested.

```
portal = ts.attach_to_portal(show_gui = portal_mode_ui, version = version)

(function) def attach_to_portal(
    show_gui: Any,
    version: Any
) -> TiaPortal

Attach to running TIA Portal instance showgui can be True or False version consists of major.minor e.g. "17.0"
*Returns* -> TiaPortal TIA Portal instance

parameters type                description
show_gui    bool    Set if gui should be displayed or not, by default False
version     string  consists of major.minor e.g. 17.0, by default selects latest installed Tia Portal
portal = siemens_tiaportal_scripting.attach_to_portal(show_gui = True, version = "18.0")
```

1.6. Scripts

The folder "Scripts" contains example scripts which are using TIA Scripting Python to interact with TIA Portal.

Script	Description
attacher.py	Attaches to a running TIA Portal instance (with user interface), retrieves the opened project, prints the names of all available PLCs, and opens the device editor of the first PLC.

Script	Description
export_hmi.py	Exports HMI-related data from the TIA Portal project.
export_library.py	Exports library elements from the TIA Portal project.
export_plc.py	Enumerates all software elements of a PLC and exports them to the configured folder.
importer.py	Opens a TIA Portal project and imports all exported PLC elements (data types, technology objects, program blocks, PLC tags, watch tables) from a specified directory into each PLC in the project.
get_products.py	Lists all installed Siemens TIA Portal product bundles and their products, including name, release, and version. Also lists all installed products individually.

1.7. Create an Executable from your script

You can convert your Python script into a standalone Windows executable (.exe) using PyInstaller. This has several benefits:

- **No Python installation required for end users:** The executable can be run on any compatible Windows system, even if Python is not installed.
- **Simplified distribution:** You can share a single .exe file instead of requiring users to set up a Python environment and dependencies.
- **Protection of source code:** The Python source code is bundled and not directly exposed.

1.7.1. Steps to create an executable

1. **Install PyInstaller** Open a command prompt and run:

```
pip install pyinstaller
```

2. **Prepare your script** Make sure your script (for example, `demo.py`) is ready and located in the folder where you want to build the executable. Replace `demo.py` with the name of your own script.

3. **Run PyInstaller** In the same folder as your script, run:

```
pyinstaller --onefile demo.py
```

- `--onefile` tells PyInstaller to bundle everything into a single .exe file.
- `demo.py` is your script to be converted. Replace it with your actual script name.
- The resulting executable will be placed in the `dist` subfolder.

(Optional: Use the `-p` option to add additional paths for dependencies, e.g., `-p D:\PyLibs\` if you have custom libraries.)

4. **Find your executable** After the process completes, look in the `dist` folder for your new `.exe` file (e.g., `dist\demo.exe`).

When building a Python executable, it's important to ensure that local binaries (such as `siemens_tia_scripting.pyd`) are used instead of any globally installed packages. The following instructions will guide you through the process of achieving this using **PyInstaller**.

To ensure your Python script loads local binaries, include the following code. This will dynamically locate the `.pyd` file bundled with the executable:

```
import importlib.util
import os
import sys

# Get the file from the location extracted by PyInstaller
def get_file_path(file_name):
    if hasattr(sys, '_MEIPASS'):
        return os.path.join(sys._MEIPASS, file_name)
    return os.path.join(os.path.abspath("."), file_name)

# Import this specific package, bypassing globally installed ones
spec = importlib.util.spec_from_file_location("siemens_tia_scripting",
get_file_path("siemens_tia_scripting.pyd"))
ts = importlib.util.module_from_spec(spec)
spec.loader.exec_module(ts)
```

`get_file_path()`: This function ensures that the correct local version of the binary file is used, especially when the script is packaged into an executable.

The code dynamically loads the `.pyd` binary, ensuring that local binaries are used, bypassing any globally installed packages.

You can use the following PyInstaller command to build the executable directly from the command line:

```
pyinstaller -F /mytiascript.py --workpath /temp -n MyTiaExecutable --clean --distpath ./output  
--add-data "../dep/:"
```

- **-F**: Creates a single standalone executable.
- **/mytiascript.py**: The path to your Python script that needs to be compiled.
- **--workpath /temp**: Specifies the temporary working directory during the build process (in this case, `../temp`).
- **-n MyTiaExecutable**: Specifies the name of the resulting executable.
- **--clean**: Cleans up any temporary files from previous builds before starting.
- **--distpath ./output**: The path where the final executable will be placed (in this case, `./output`).
- **--add-data "../dep/:"**: Adds additional data (like `.pyd` binaries) required by the executable (in this case, the contents of the `../dep/` directory).

1.8. Hierarchy

The overview shows the available models and their dependency to each other. Accessing a program block from a PLC requires that a TIA Portal connection is established, a project opened and a controller object selected.

2. Function List

The first lines of any script should import the TIA Scripting Python, whereas the previously defined system environment variable is used therefore.

```
sys.path.append(os.getenv('TIA_SCRIPTING'))  
import siemens_tia_scripting
```

If the import was successful, the appropriate methods from TIA Scripting Python can be used.

2.1. Devices - Hmi

The class represents HMI (Human Machine Interface) devices in TIA Portal. It allows controlling and interacting with the HMI, including compilation and upgrading tasks.

2.1.1. get_name()

Get the name of the HMI

Returns → `str` Name of the HMI

```
hmi_name = hmi.get_name()
```

2.1.2. get_hmi_type()

Get the type of the HMI

Returns → `str` Type of the HMI

```
hmi_type = hmi.get_hmi_type()
```

2.1.3. open_device_editor()

Open the editor of the HMI-device in TIA Portal

```
hmi.open_device_editor()
```

2.1.4. compile_hardware()

Compile the hardware of the HMI

Returns → `bool` Result of the compile

Returns true if compile has errors, otherwise returns false

```
result = hmi.compile_hardware()
```

2.1.5. compile_software()

Compile the software of the HMI

Returns → `bool` Result of the compile

Returns true if compile has errors, otherwise returns false

```
result = hmi.compile_software()
```

2.1.6. upgrade_hardware(full_upgrade: bool)

Update hardware of the HMI

If `full_upgrade` is set to True, all devices will be changed to the newest available order number (Device type) and firmware:

from CPU1511F 6ES7 511-1FK00-0AB0 **V1.7** to 6ES7 511-1FK00-0AB0 **V1.8**.

With full upgrade: from CPU1511F 6ES7 511-1FK00-0AB0 **V1.7** to 6ES7 511-1FL03-0AB0 **V3.0**.

Parameters	Type	Description
full_upgrade	bool	Set if for full upgrade (latest order number and firmware version)

```
hmi.upgrade_hardware(full_upgrade = True)
```

2.1.7. **get_property(name: str)**

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.1.8. **get_properties()**

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.1.9. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.1.10. **get_identifier()**

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.1.11. **get_hmi_tag_tables(folder_path: Optional[str] = None)**

Get the list of the HMI Tag Tables

Returns → `List[HmiTagTable]` List of the HMI Tag tables

Parameters	Type	Description
folder_path	str	The group path of the item

```
hmi_tag_tables = hmi.get_hmi_tag_tables(folder_path = "group1/group2")
```

2.1.12. get_screens(folder_path: Optional[str] = None)

Get the list of the HMI Screens

Returns → `List[HmiScreen]` List of the HMI Screens

Parameters	Type	Description
folder_path	str	The group path of the item

```
hmi_screens = hmi.get_screens(folder_path = "group1/group2")
```

2.1.13. get_text_lists()

Get the list of the HMI Text Lists

Returns → `List[HmiTextList]` List of the HMI Text lists

<pre>hmi_lists = hmi.get_text_lists()</pre>

2.1.14. get_scripts(folder_path: Optional[str] = None)

Get the list of the HMI Scripts (VB Comfort), (JavaScript Unified)

Returns → `List[HmiScript]` List of the HMI Script

Parameters	Type	Description
folder_path	str	The group path of the item

```
hmi_scripts = hmi.get_scripts(folder_path = "group1/group2")
```

2.1.15. get_alarms()

Get the list of the HMI Alarms

Returns → `List[HmiAlarm]` List of the HMI Alarms

<pre>hmi_alarms = hmi.get_alarms()</pre>
--

2.1.16. get_alarm_classes()

Get the list of the HMI Alarm Classes

Returns → `List[HmiAlarmClass]` List of the HMI Alarm Classes

<pre>hmi_alarms = hmi.get_alarms_classes()</pre>
--

2.1.17. get_connections()

Get the list of the HMI Connection

Returns → `List[HmiAlarm]` List of the HMI Connection

<pre>connections = hmi.get_connections()</pre>
--

2.1.18. get_cycles()

Get the list of the HMI Cycles

Returns → `List[HmiCycle]` List of the HMI Cycles

<pre>cycles = hmi.get_cycles()</pre>

2.1.19. get_graphic_lists()

Get the list of the HMI GraphicLists

Returns → `List[HmiGraphicList]` List of the HMI GraphicLists

```
graph_lists = hmi.get_graphic_lists()
```

2.1.20. get_global_screen_elements()

Get the Hmi screen global elements

Returns → `HmiScreen` Hmi screen global elements

```
global_elements = hmi.get_global_screen_elements()
```

2.1.21. get_screen_overview()

Get the Hmi screen overview

Returns → `HmiScreen` Hmi screen overview

```
screen_overview = hmi.get_screen_overview()
```

2.1.22. get_slide_in_screens(folder_path: Optional[str] = None)

Get the list of the HMI SlideIn Screens

Returns → `List[HmiScreen]` List of the HMI Screens

Parameters	Type	Description
folder_path	str	The group path of the item

```
hmi_slide_in_screens = hmi.get_slide_in_screens(folder_path = "group1/group2")
```

2.1.23. import_hmi_tags(import_root_directory: str, target_folder_path: Optional[str] = None)

Import tags from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the HMI's group structure where the imported folder hierarchy will be recreated

```
hmi.import_hmi_tags(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Tags")
```

2.1.24. import_connections(import_root_directory: str)

Import connections from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
hmi.import_hmi_tags(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Connections")
```

2.1.25. import_cycles(import_root_directory: str)

Import cycles from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
hmi.import_cycles(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Cycles")
```

2.1.26. **import_scripts(import_root_directory: str, target_folder_path: Optional[str] = None)**

Import scripts from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the HMI's group structure where the imported folder hierarchy will be recreated

```
hmi.import_scripts(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Scripts")
```

2.1.27. **import_text_lists(import_root_directory: str)**

Import Text lists from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
hmi.import_text_lists(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Text lists")
```

2.1.28. **import_graphic_lists(import_root_directory: str)**

Import graphic lists from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
hmi.import_graphic_lists(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Graphics")
```

2.1.29. **import_screens(import_root_directory: str, target_folder_path: Optional[str] = None)**

Import screens from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the HMI's group structure where the imported folder hierarchy will be recreated

```
hmi.import_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\Screens")
```

2.1.30. **import_popup_screens(import_root_directory: str, target_folder_path: Optional[str] = None)**

Import popup screens from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

Parameters	Type	Description
target_folder_path	str	Optional: The target base path within the HMI's group structure where the imported folder hierarchy will be recreated

```
hmi.import_popup_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\PopUpScreens")
```

2.1.31. import_template_screens(import_root_directory: str, target_folder_path: Optional[str] = None)

Import template screens from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the HMI's group structure where the imported folder hierarchy will be recreated

```
hmi.import_template_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\TemplateScreens")
```

2.1.32. import_slidein_screens(import_root_directory: str)

Import slide in screens from a directory to the HMI

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
hmi.import_slidein_screens(import_root_directory = "C:\\ws\\importfolder\\HMI_1\\SlideinScreens")
```

2.1.33. import_global_elements(import_file: str)

Import global elements screen to the HMI

Parameters	Type	Description
import_file	str	Global elements import file

```
hmi.import_global_elements(import_file = "C:\\ws\\importfolder\\HMI_1\\GlobalElements.xml")
```

2.1.34. import_screen_overview(import_file: str)

Import screen overview to the HMI

Parameters	Type	Description
import_file	str	Screen overview import file

```
hmi.import_screen_overview(import_file = "C:\\ws\\importfolder\\HMI_1\\ScreenOverview.xml")
```


2.2. Devices - Plc

This represents a PLC (Programmable Logical Controller). It allows for controlling and interacting with the PLC, including downloading, comparing online and offline states, and retrieving information about PLC elements.

2.2.1. get_name()

Get the name of the PLC

Returns → `str` Name of the PLC

```
plc_name = plc.get_name()
```

2.2.2. open_device_editor()

Open the editor of the PLC-device in TIA Portal

```
plc.open_device_editor()
```

2.2.3. get_online_state()

Get the current online state of the PLC

Returns → `str` Online state

```
plc.get_online_state()
```

2.2.4. go_offline()

Go offline with the PLC

```
plc.go_offline()
```

2.2.5. go_online(pc_interface_type: str, pc_interface_name: str, target_interface: str)

Go online with the PLC

Returns → `str` Online status

Parameters	Type	Description
pc_interface_type	str	Configuration mode
pc_interface_name	str	PC Interface name
target_interface	str	API_V21: IP address or target interface, under v21 only target interface

```
plc.go_online(pc_interface_type = "PN/IE", pc_interface_name = "PLCSIM", target_interface = "1X1")
```

2.2.6. download(pc_interface_type: str, pc_interface_name: str, target_interface: str)

Download the PLC

Returns → `str` Result of the download

Parameters	Type	Description
pc_interface_type	str	Configuration mode
pc_interface_name	str	PC Interface name
target_interface	str	IP address

```
result = plc.download(pc_interface_type = "PN/IE", pc_interface_name = "PLCSIM",
target_interface = "1 X1")
```

2.2.7. download_to_memory_card(download_directory: str, is_plcsim_advanced: Optional[bool] = None)

Download the PLC-configuration to memory card

Returns → `str` Result of the download

Parameters	Type	Description
download_directory	str	Download directory
is_plcsim_advanced	bool	True: Plc sim advanced download is configured, by default false for the CPU download

```
result = plc.download_to_memory_card(download_directory = "E:\\", is_plcsim_advanced = True)
```

2.2.8. get_plc_tag_tables(folder_path: Optional[str] = None)

Get the list of the PLC Tag Tables

Returns → `List[PlcTagTable]` List of the PLC Tag tables

Parameters	Type	Description
folder_path	str	The group path of the item

```
plc_tag_tables = plc.get_plc_tag_tables(folder_path = "group1/group2")
```

2.2.9. update_module_description()

Update the module description of the PLC

Returns → `bool` Result state

```
result = plc.update_module_description()
```

2.2.10. compile_hardware()

Compile the hardware of the PLC

Returns → `bool` Result of the compile

Returns true if compile has errors, otherwise returns false

```
result = plc.compile_hardware()
```

2.2.11. compile_software()

Compile the software of the PLC

Returns → `bool` Result of the compile

Returns true if compile has errors, otherwise returns false

```
result = plc.compile_software()
```

2.2.12. upgrade_hardware(full_upgrade: bool)

Upgrade the hardware the PLC

If `full_upgrade` is set to True, all devices will be changed to the newest available order number (Device type) and firmware:

from CPU1511F 6ES7 511-1FK00-0AB0 V1.7 to 6ES7 511-1FK00-0AB0 V1.8.

With full upgrade: from CPU1511F 6ES7 511-1FK00-0AB0 V1.7 to 6ES7 511-1FL03-0AB0 V3.0.

Parameters	Type	Description
full_upgrade	bool	Set if for full upgrade (latest order number and firmware version)

```
plc.upgrade_hardware(full_upgrade = True)
```

2.2.13. compare_to_online()

Compare actual PLC state to the online state

Returns → bool Result of the compare

Returns false if folders are Identical, otherwise returns true

```
result = plc.compare_to_online()
```

2.2.14. get_program_blocks(folder_path: Optional[str] = None)

Get the list of program blocks

Returns → List[ProgramBlock] List of the program blocks

Parameters	Type	Description
folder_path	str	The group path of the item

```
blocks = plc.get_program_blocks(folder_path = "group1/group2")
```

2.2.15. get_system_blocks()

Get the list of system blocks

Returns → List[SystemBlock] List of the system blocks

```
blocks = plc.get_system_blocks()
```

2.2.16. get_user_data_types(folder_path: Optional[str] = None)

Get the list of PLC data types

Returns → List[UserDataTypes] List of the PLC data types

Parameters	Type	Description
folder_path	str	The group path of the item

```
udts = plc.get_user_data_types(folder_path = "group1/group2")
```

2.2.17. get_external_sources(folder_path: Optional[str] = None)

Get the list of external source files

Returns → List[ExternalSource] List of the external source files of the PLC

Parameters	Type	Description
folder_path	str	The group path of the item

```
ext_sources = plc.get_external_sources()
```

2.2.18. get_force_tables()

Get the list of force tables

Returns → `List[ForceTable]` List of the force tables of the PLC

```
tables = plc.get_force_tables()
```

2.2.19. **get_watch_tables(folder_path: Optional[str] = None)**

Get the list of watch tables

Returns → `List[WatchTable]` List of the watch tables of the PLC

Parameters	Type	Description
folder_path	str	The group path of the item

```
tables = plc.get_watch_tables()
```

2.2.20. **get_technology_objects(folder_path: Optional[str] = None)**

Get the list of technology objects

Returns → `List[TechnologyObject]` List of the technology objects of the PLC

Parameters	Type	Description
folder_path	str	The group path of the item

```
tos = plc.get_technology_objects()
```

2.2.21. **get_software_units()**

Get the list of software units

Returns → `List[SoftwareUnit]` List of the software units

```
software_units = plc.get_software_units()
```

2.2.22. **get_safety_administration()**

Get Safety administration of the PLC

Returns → `SafetyAdministration` Safety Administration of the PLC

```
sa = plc.get_safety_administration()
```

2.2.23. **import_blocks(import_root_directory: str, target_folder_path: Optional[str] = None)**

Import program blocks from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the PLC's group structure where the imported folder hierarchy will be recreated

```
plc.import_blocks(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Program blocks")
```

2.2.24. **import_plc_tags(import_root_directory: str, target_folder_path: Optional[str] = None)**

Import tags from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the PLC's group structure where the imported folder hierarchy will be recreated

```
plc.import_plc_tags(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC tags")
```

2.2.25. import_data_types(import_root_directory: str, target_folder_path: Optional[str] = None)

Import user data types from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the PLC's group structure where the imported folder hierarchy will be recreated

```
plc.import_data_types(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC data types")
```

2.2.26. import_technology_objects(import_root_directory: str, target_folder_path: Optional[str] = None)

Import technology objects from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the PLC's group structure where the imported folder hierarchy will be recreated

```
plc.import_technology_objects(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Technology objects")
```

2.2.27. import_watch_tables(import_root_directory: str, target_folder_path: Optional[str] = None)

Import watch tables from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder
target_folder_path	str	Optional: The target base path within the PLC's group structure where the imported folder hierarchy will be recreated

```
plc.import_watch_tables(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Watch and force tables")
```

2.2.28. create_software_unit(name: str)

Create software unit on the PLC

Parameters	Type	Description
name	str	Software unit new name

```
plc.create_software_unit(name = "Unit 1")
```

2.2.29. **safety_print(print_file: str)**

Create a safety printout of the PLC and print it to a file

If the file already exists, it is overwritten.

Returns → `bool` Returns true on success

Parameters	Type	Description
print_file	str	Full path of the printout file

```
plc.safety_print(print_file = "C:\\ws\\safetyprint\\F_PLC_Printout.pdf")
```

2.2.30. **get_property(name: str)**

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.2.31. **get_properties()**

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.2.32. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.2.33. **get_identifier()**

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifiser()
```

2.3. Enums

2.3.1. PortalMode

enum element	value
WithGraphicalUserInterface	0
WithoutGraphicalUserInterface	1
AnyUserInterface	2

2.3.2. UmacUserMode

enum element	value
Project	0
Global	1

2.3.3. ExportFormats

enum element	value
SimaticML	0
ExternalSource	1
SimaticSD	2

2.3.4. ExportOptions

enum element	value
WithDefaults	0
Nan	1
WithReadOnly	2

2.3.5. CleanUpMode

enum element	value
PreserveDefaultVersionOfUnusedTypes	0
DeleteUnusedTypes	1

2.3.6. LibraryExportOptions

enum element	value
Nan	0
WithLibraryVersionInfoFile	1
OnlyLibraryVersionInfoFile	2

2.3.7. HarmonizeOptions

enum element	value
HarmonizePathsAndNames	0
HarmonizePaths	1
HarmonizeNames	2

2.3.8. DependenciesMode

enum element	value
DoNotAutomaticallyCreateOrReleaseDependencies	0
AutomaticallyCreateOrReleaseDependenciesIfRequired	1

2.4. Global - Global

Functions to start or attach to TIA Portal are available to get the TIA Portal instance.

2.4.1. **open_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)**

Open a new TIA Portal instance Version-string in format major.minor e.g. "18.0"

Returns → `Portal` TIA Portal instance

Parameters	Type	Description
portal_mode	Enums.PortalMode	With or without user-interface
version	str	Version string of TIA Portal to be used in format major.minor e.g. 18.0, (Latest installed TIA Portal version by default)

Example usage

```
portal = siemens_tia_scripting.open_portal(portal_mode =
siemens_tia_scripting.Enums.PortalMode.WithGraphicalUserInterface, version = "18.0")
```

2.4.2. **attach_portal(portal_mode: Optional[Enums.PortalMode] = None, version: Optional[str] = None)**

Attach to running TIA Portal instance Version-string in format major.minor e.g. "18.0"

Returns → `Portal` TIA Portal instance

Parameters	Type	Description
portal_mode	Enums.PortalMode	With or without user-interface
version	str	Version string of TIA Portal to be used in format major.minor e.g. 18.0, (Latest installed TIA Portal version by default)

```
portal = siemens_tia_scripting.attach_portal(portal_mode =
Enums.PortalMode.WithGraphicalUserInterface, version = "18.0")
```

2.4.3. **open_attach_project(project_file_path: str, portal_mode: Optional[Enums.PortalMode] = None, server_project_view: Optional[bool] = None)**

Attach to running TIA Portal instance with already open project Or opens the project with a new instance of fitting TIA Portal version

Returns → `Project` TIA Project instance

Parameters	Type	Description
project_file_path	str	Full path of the project file
portal_mode	Enums.PortalMode	With or without user-interface
server_project_view	str	Default: false. If parameter set to true, open in server project view

```
project = siemens_tia_scripting.open_attach_project(project_file_path =
"C:\\ws\\testproj\\testproj.ap17", portal_mode =
siemens_tia_scripting.Enums.PortalMode.WithGraphicalUserInterface)
```

2.4.4. **get_installed_bundles()**

Get a list of installed TIA Portal bundles

Returns → `List[ProductBundle]` List of installed bundles

```
product_bundles = siemens_tia_scripting.get_installed_bundles()
```

2.4.5. `get_installed_products()`

Get a list of installed TIA Portal products

Returns → `List[Product]` List of installed products

```
products = siemens_tia_scripting.get_installed_products()
```

2.4.6. `set_umac_credentials(user_name: str, user_password: str, user_type: Enums.UmacUserMode)`

Set UMAC Credentials which will be used for protected libraries or projects

Parameters	Type	Description
<code>user_name</code>	<code>str</code>	User name
<code>user_password</code>	<code>str</code>	Password of the user
<code>user_type</code>	<code>Enums.UmacUserMode</code>	Project or global user

```
siemens_tia_scripting.set_umac_credentials(user_name = "admin", user_password = "Password123", user_type = Enums.UmacUserMode.Project)
```

2.4.7. `encrypt_umac_config(umac_file_path: str, secret: str, secret_env_name: str)`

Encrypt UMAC and UMC configuration from the TIA Portal project with provided secret

parameters	type	description
<code>umac_file_path</code>	<code>str</code>	Full path of the umac config file
<code>secret</code>	<code>str</code>	Secret for the encryption
<code>secret_env_name</code>	<code>str</code>	Name of the environment variable where secret value is stored

```
siemens_tia_scripting.encrypt_umac_config(umac_file_path = "C:\\ws\\exportfolder\\exportUMAC.json", secret = "mySecret")
```

2.4.8. `set_logging(path: str, console: bool)`

Set logging output path or console (stdout) output

Parameters	Type	Description
<code>path</code>	<code>str</code>	Fullpath of the log file
<code>console</code>	<code>bool</code>	Optional: True for Enable console output, False to disable the standard output to console window

```
siemens_tia_scripting.set_logging(path = "c:\\ws\\tiascripting.log", console = False)
```

2.5. Global - Portal

The Portal-object represents an open TIA Portal instance and provides functionality to manage projects and libraries within TIA Portal.

2.5.1. get_process_id()

Get the TIA Portal process ID

Returns → `int` TIA Portal process id

```
id = portal.get_process_id()
```

2.5.2. open_project(project_file_path: str, server_project_view: Optional[bool] = None)

Open a TIA Portal project or local session - optional in server view

Returns → `Project` TIA Portal project

Parameters	Type	Description
project_file_path	str	Full path of the project file
server_project_view	str	Default: false. If parameter set to true, open in server project view

```
project = portal.open_project(project_file_path = "C:\\ws\\testproj\\testproj.ap17")
```

2.5.3. open_project_with_copy(project_file_path: str, target_directory_path: str, delete_existing_project: bool)

Open a TIA Portal project or local session in separate folder

Returns → `Project` TIA Portal project

Parameters	Type	Description
project_file_path	str	Full path of the project file
target_directory_path	str	Temporary path where the project should be copied before opened
delete_existing_project	bool	Defines if the temporary project should be deleted

```
project = portal.open_project_with_copy(project_file_path = "C:\\ws\\testproj\\testproj.ap17",
target_directory_path = "C:\\ws\\temp" , delete_existing_project = True)
```

2.5.4. retrieve_archive(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)

Retrieve a TIA Portal project archive

Returns → `Project` TIA Portal project

Parameters	Type	Description
target_directory_path	str	Temporary path where project should be copied before opened
archive_file_path	str	Full path of the archived project file
delete_existing_project	bool	Defines if the temporary project should be deleted

```
project = portal.retrieve_archive(archive_file_path = "C:\\ws\\testproj\\testproj.zap17",
target_directory_path = "C:\\ws\\temp" , delete_existing_project = True)
```

2.5.5. **create_project(target_directory_path: str, project_name: str, delete_existing_project: bool)**

Create a new TIA Portal project

Returns → `Project` TIA Portal project

Parameters	Type	Description
target_directory_path	str	Temporary path where project should be copied before opened
project_name	str	Name of the new project
delete_existing_project	bool	Defines if the temporary project should be deleted

```
project = portal.create_project(target_directory_path = "C:\\ws\\temp" , project_name = "MyNewProject" delete_existing_project = True)
```

2.5.6. **get_project()**

Get opened TIA Portal project of the current TIA Portal instance

Returns → `Project` TIA Portal project

```
project = portal.get_project()
```

2.5.7. **get_global_library_infos()**

Get a list of global library infos of the current TIA Portal instance

Returns → `List[GlobalLibraryInfo]` List of the global library information

```
global_lib_infos = portal.get_global_library_infos()
for info in global_lib_infos:
    ...
```

2.5.8. **get_global_library(library_name: str)**

Get global library by name

Returns → `GlobalLibrary` Global Library

Parameters	Type	Description
library_name	str	Name of the global library

```
global_lib = portal.get_global_library(library_name = "GlobalLib1")
```

2.5.9. **open_global_library(library_path: str)**

Open global library by path

Returns → `GlobalLibrary` Global Library

Parameters	Type	Description
library_path	str	Full path of the global library

```
global_lib = portal.open_global_library(library_path = "C:\\ws\\testlib\\testlib.al17")
```

2.5.10. **open_global_library_with_copy(target_directory_path: str, library_path: str, delete_existing_project: bool)**

Open global library from a copy

Returns → `GlobalLibrary` Global Library

Parameters	Type	Description
target_directory_path	str	Temporary path where the library should be copied before opened
library_path	str	Full path of the global library
delete_existing_project	bool	Defines if the temporary project should be deleted

```
global_lib = portal.open_global_library_with_copy(target_directory_path = "C:\\ws\\temp",
library_path = "C:\\ws\\testlib\\testlib.al17", delete_existing_project = True)
```

2.5.11. create_global_library(target_directory_path: str, library_name: str, delete_existing_project: bool)

Create a new TIA Portal project

Returns → `GlobalLibrary` Global Library

Parameters	Type	Description
target_directory_path	str	Temporary path where the library should be copied before opened
library_name	str	Name of the new global library
delete_existing_project	bool	Defines if the temporary library should be deleted

```
global_lib = portal.create_global_library(target_directory_path = "C:\\ws\\temp" , library_name
= "MyLib1" delete_existing_project = True)
```

2.5.12. retrieve_archive_library(target_directory_path: str, archive_file_path: str, delete_existing_project: bool)

Retrieve a TIA Portal library archive

Returns → `GlobalLibrary` Global Library

Parameters	Type	Description
target_directory_path	str	Temporary path where library should be copied before opened
archive_file_path	str	Full path of the archived library file
delete_existing_project	bool	Defines if the temporary project should be deleted

```
global_lib = portal.retrieve_archive_library(target_directory_path = "C:\\ws\\temp",
archive_file_path = "C:\\ws\\testproj\\testlib.za17", delete_existing_project = True)
```

2.5.13. close_portal()

Attempts to close the TIA Portal instance (requires all projects to be saved or changes explicitly discarded)

```
portal.close_portal()
```

2.5.14. show_online_finger_prints(mode: str, pci_interface: str, ip_address: str)

Show fingerprints of the PLC

Parameters	Type	Description
mode	str	Configuration mode
pci_interface	str	PC Interface name
ip_address	str	IP adress

```
portal.show_online_finger_prints(mode = "PN/IE", pci_interface = "PLCSIM", ip_address = "192.168.0.10")
```

2.5.15. **get_project_servers(server: Optional[str] = None)**

Get a list of TIA Project-Servers found by host-filter

Returns → `List[ProjectServer]` List of TIA Project-Servers

Parameters	Type	Description
server	str	Server Name or URL

```
portal.get_project_servers(server = "CompanyServer")
```

2.5.16. **detach()**

Detach from the TIA Portal instance

```
portal.detach()
```

2.5.17. **add_project_server(url: str, name: str)**

Add new TIA Project-Server

Returns → `ProjectServer` TIA Project-Server

Parameters	Type	Description
url	str	URL of the TIA Project-Server
name	str	Name of new server

```
portal.add_project_server(url = "https://project.server.net:8735/", name = "MyNewServer")
```

2.6. Global - Product

The Product-object represents an object of installed Siemens product software.

2.6.1. `get_name()`

Get the name of the TIA Portal product

Returns → `str` Name of the TIA Portal product

```
product_name = product.get_name()
```

2.6.2. `get_release()`

Get the release of the TIA Portal product

Returns → `str` Release tag of the TIA Portal product

```
product_release = product.get_release()
```

2.6.3. `get_version()`

Get the version of the TIA Portal product

Returns → `str` Version of the TIA Portal product

```
product_version = product.get_version()
```

2.7. Global - ProductBundle

The ProductBundle-object represents an object of installed Siemens bundle software.

2.7.1. **get_title()**

Get the title of the TIA Portal Bundle

Returns → `str` Title of the Siemens bundle

```
bundle_title = bundle.get_title()
```

2.7.2. **get_release()**

Get the release of the TIA Portal Bundle

Returns → `str` Release of the Siemens bundle

```
bundle_release = bundle.get_release()
```

2.7.3. **get_products()**

Get the list of products of the TIA Portal Bundle

Returns → `List[Product]` List of Siemens products

```
bundle_products = bundle.get_products()
```


2.8. HMI Data - HmiAlarm

The class represents an HMI Alarm in TIA Portal. It provides methods for getting information about the HMI Alarm, retrieving properties.

2.8.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.8.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.8.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.8.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.8.5. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.9. HMI Data - HmiAlarmClass

The class represents an HMI Alarm Class in TIA Portal. It provides methods for getting information about the HMI Alarm Class, retrieving properties.

2.9.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.9.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.9.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.9.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.9.5. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.10. HMI Data - HmiConnection

The class represents an HMI Connection in TIA Portal. It provides methods for getting information about the HMI Connection, retrieving properties.

2.10.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.10.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.10.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.10.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.10.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.10.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.11. HMI Data - HmiCycle

The class represents an HMI Cycle in TIA Portal. It provides methods for getting information about the HMI Cycle, retrieving properties.

2.11.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.11.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.11.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.11.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.11.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.11.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.12. HMI Data - HmiGraphicList

The class represents an HMI Graphic List in TIA Portal. It provides methods for getting information about the HMI Graphic List, retrieving properties.

2.12.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.12.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.12.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.12.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.12.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =  
Enums.ExportOptions.WithDefaults)
```

2.12.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```


2.13. HMI Data - HmiScreen

The class represents an HMI Screen in TIA Portal. It provides methods for getting information about the HMI Screen, retrieving properties.

2.13.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.13.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.13.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.13.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.13.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.13.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.14. HMI Data - HmiScript

The class represents an HMI Script in TIA Portal. It provides methods for getting information about the HMI Script, retrieving properties.

2.14.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.14.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.14.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.14.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.14.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.14.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.15. HMI Data - HmiTag

The class represents an HMI Tag in TIA Portal. It provides methods for getting information about the HMI Tag, retrieving properties.

2.15.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.15.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.15.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.15.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.15.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.15.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.16. HMI Data - HmiTagTable

The class represents an HMI Tag Table in TIA Portal. It provides methods for getting information about the HMI Tag Table, retrieving properties.

2.16.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.16.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.16.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.16.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.16.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.16.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```


2.17. HMI Data - HmiTextList

The class represents an HMI Text List in TIA Portal. It provides methods for getting information about the HMI Text List, retrieving properties.

2.17.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.17.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.17.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.17.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.17.5. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options =
Enums.ExportOptions.WithDefaults)
```

2.17.6. get_identifier()

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.18. Library - GlobalLibrary

The class provides methods to interact with TIA Portal global libraries. It allows you to get information about the library, save it, get the author, check if it's modified and read-only state, and perform various operations on the library.

2.18.1. get_name()

Get the name of the global library

Returns → `str` Name of the global library

```
name = global_lib.get_name()
```

2.18.2. save()

Save the global library

```
global_lib.save()
```

2.18.3. get_author()

Get author of the global library

Returns → `str` Name of the author

```
author = global_lib.get_author()
```

2.18.4. get_path()

Get the path of the global library

Returns → `str` Full path of the library

```
path = global_lib.get_path()
```

2.18.5. is_modified()

Check if the global library was modified

Returns → `bool` State if library was modified

```
state = global_lib.is_modified()
```

2.18.6. is_read_only()

Check if the global library is readonly

Returns → `bool` State if global library is readonly

```
state = global_lib.is_read_only()
```

2.18.7. update_library(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None, library_name: Optional[str] = None)

Update target library with type(s) from source, if no target library selected the project library will be updated

Parameters	Type	Description
update_mode	integer (enum)	[1: 'ForceSetAnyUpdatedVersionAsDefault', 2: 'NoDefaultVersionChange', 3: 'SetOnlyHigherUpdatedVersionAsDefault']

Parameters	Type	Description
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
conflict_mode	integer (enum)	[1: 'CancelIfStructureConflicts', 2: 'RetainStructure', 3: 'UpdateStructure']
type_guids	List[str]	Optional: This option should be used to select specific source type or types. If empty, all types in project library are selected
library_name	str	Optional: This option should be used when a specific global library has to be updated. If no name provided, the project library of the current project will be selected

```
global_lib.update_library(update_mode = 1, delete_mode = 1, conflict_mode = 3, type_guids = ["93826aed-7eac-4380-b4f4-f58e2707c862", "1fb57e17-f627-45f7-b7b8-644954b8888b"], library_name = "MyGlobalLibrary")
```

2.18.8. update_project(update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None)

Update current project with type(s) from global library

Parameters	Type	Description
update_mode	integer (enum)	[1: 'ForceSetAnyUpdatedVersionAsDefault', 2: 'NoDefaultVersionChange', 3: 'SetOnlyHigherUpdatedVersionAsDefault']
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
conflict_mode	integer (enum)	[1: 'CancelIfStructureConflicts', 2: 'RetainStructure', 3: 'UpdateStructure']
type_guids	List[str]	Optional: This option should be used to select specific source type or types. If empty, all types in project library are selected

```
global_lib.update_project(update_mode = 1, delete_mode = 1, conflict_mode = 3, type_guids = ["93826aed-7eac-4380-b4f4-f58e2707c862", "1fb57e17-f627-45f7-b7b8-644954b8888b"])
```

2.18.9. harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)

Harmonize type instances in project using global library. If type_guids provided, only the selected types will be used for update

Parameters	Type	Description
harmonize_options	integer (enum)	[0: 'HarmonizePathsAndNames', 1: 'HarmonizePaths', 2: 'HarmonizeNames']
type_guids	List[str]	Optional: This option should be used to select specific source type or types. If empty, all types in project library are selected

```
global_lib.harmonize_project(harmonize_options: 0, type_guids: ["779c66b7-8fe2-46c0-a8a9-7da387b2521b", "bb856213-036d-47e4-9e8b-129e3c1606f6"])
```

2.18.10. archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)

Archive the TIA Portal library

Parameters	Type	Description
target_directory_path	str	Folder path where archive should be placed
archive_name	str	Name of the archive file
delete_existing_archive	bool	Set if the output file should be deleted first

```
global_lib.archive(target_directory_path = "C:\\ws\\newfolder", archive_name = "testglobalib", delete_existing_archive = True)
```

2.18.11. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → **str** Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.18.12. get_properties()

Get all properties of the object

Returns → **List[str]** Names of the attributes

```
properties = tiap_object.get_properties()
```

2.18.13. set_property(name:str, value: str)

Set the property of the object

Returns → **int** Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.18.14. clean_up(folder_path: Optional[str] = None)

Performs the cleanup of the GlobalLibrary, without deleting the complete types

Parameters	Type	Description
folder_path	str	Optional: The target base path within the project library where the imported folder hierarchy will be recreated

```
global_lib.clean_up()
```

2.18.15. get_types()

Get the library types of the folder

Returns → `List[LibraryType]` List of types

```
types = global_lib.get_types()
```

2.18.16. find_library_type(library_type_name: str)

Find the library type by name

Returns → `LibraryType` Library type

Parameters	Type	Description
library_type_name	str	Name of the library type

```
type = global_lib.find_library_type(library_type_name = "MyType")
```

2.18.17. close_global_library()

Close the global library instance

```
global_lib.close_global_library()
```

2.18.18. create_folder(folder_path: str)

Create new folder at specified path in the library

Parameters	Type	Description
folder_path	str	Optional: The target base path within the global library where the imported folder hierarchy will be recreated

```
global_lib.create_folder(folder_path = "myFolder/subFolderExample")
```

2.18.19. delete_folder(folder_path: str)

Delete specified folder in global library

Parameters	Type	Description
folder_path	str	Optional: The target base path within the global library where the imported folder hierarchy will be recreated

```
global_lib.delete_folder(folder_path = "myFolder/subFolderExample")
```

2.19. Library - GlobalLibraryInfo

The class represents information about a global library in the context of TIA Portal.

2.19.1. get_name()

Get the name of the global library info

Returns → `str` Name of the global library info

```
name = global_lib_info.get_name()
```

2.19.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.19.3. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.19.4. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.20. Library - LibraryType

The class represents a library type in the context of TIA Portal.

2.20.1. get_name()

Get the name of the library type

Returns → `str` Name of the type

```
name = lib_type.get_name()
```

2.20.2. get_author()

Get the author of the library type

Returns → `str` Author of the type

```
author = lib_type.get_author()
```

2.20.3. get_guid()

Get the guid of the library type

Returns → `str` Guid of the type

```
guid = lib_type.get_guid()
```

2.20.4. get_versions()

Get the versions of the library type

Returns → `List[LibraryTypeVersion]` Versions of the type

```
versions = lib_type.get_versions()
```

2.20.5. find_version(version: str)

Find the version of the library type

Returns → `LibraryTypeVersion` Version des Bibliothekstyp

```
typeversion = lib_type.find_version(version = "1.0.0")
```

2.20.6. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.20.7. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes


```
properties = tiap_object.get_properties()
```

2.20.8. **set_property(name:str, value: str)**

Set the property of the object

Returns → **int** Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.20.9. **get_path_full()**

Get group path for an object until project

Returns → **str** group full path

```
group_path = lib_type.get_path_full()
```

2.20.10. **get_path()**

Get group path for an object until parent system folder

Returns → **str** group path

```
group_path = lib_type.get_path_full()
```

2.20.11. **get_comment()**

Get text value from the comment. Language is selected by the current editing language.

Returns → **str** Comment text

```
comment = lib_type.get_comment()
```

2.20.12. **get_identifier()**

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.21. Library - LibraryTypeFolder

The class represents a folder containing library types and subfolders in the context of TIA Portal.

2.21.1. get_name()

Get the name of the folder

Returns → `str` Name of the folder

```
name = lib_folder.get_name()
```

2.21.2. get_folders()

Get the library type folders

Returns → `List[LibraryTypeFolder]` List of folders

```
folders = lib_folder.get_folders()
```

2.21.3. get_types()

Get the library types of the folder

Returns → `List[LibraryType]` List of types

```
types = lib_folder.get_types()
```

2.21.4. find_library_type(library_type_name: str)

Find the library type by name

Returns → `LibraryType` Library type

Parameters	Type	Description
library_type_name	str	Name of the library type

```
type = lib_folder.find_library_type(library_type_name = "MyType")
```

2.21.5. find_folder(folder_name: str)

Find the library type folder from the subfolders

Returns → `LibraryTypeFolder` Library type folder

Parameters	Type	Description
folder_name	str	Name of the subfolder

```
folder = lib_folder.find_folder(folder_name = "MyFolder")
```

2.22. Library - LibraryTypeVersion

The class represents a version of a library type in the context of TIA Portal.

2.22.1. get_author()

Get the author of the library type version

Returns → `str` Author

```
author = projectlib.get_author()
```

2.22.2. get_guid()

Get the GUID of the library type version

Returns → `str` GUID

```
guid = lib_type_version.get_guid()
```

2.22.3. get_version_number()

Get the version number of the library type version

Returns → `str` version number

```
version_number = lib_type_version.get_version_number()
```

2.22.4. get_modified_date()

Get the modification date of the library type version

Returns → `str` Modification date

```
date = lib_type_version.get_modified_date()
```

2.22.5. get_state()

Get the state of the library type version

Returns → `str` State of the library type version

```
state = lib_type_version.get_state()
```

2.22.6. get_type_object()

Get the type of the library type version

Returns → `LibraryType` Library type object

```
lib_type = lib_type_version.get_type_object()
```

2.22.7. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.22.8. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.22.9. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.22.10. find_instances(device_name: Optional[str] = None)

Get the list of instances of library type version

Returns → `List[object]` List of the object instances

Parameters	Type	Description
device_name	str	Optional: Name of the device where to search instance(e.g., PLC or HMI)

```
blocks = lib_type_version.find_instances()
```

2.22.11. instantiate(device_name: str, software_unit_name: Optional[str] = None, folder_path: Optional[str] = None)

Create an Instance in the PLC or HMI of the current type version

Parameters	Type	Description
device_name	str	Name of the PLC Software or HMI Software e.g. HMI_RT_1
software_unit_name	str	Optional: Name of the PLC Software Unit
folder_path	str	Optional: group or folders structure e.g. group1/group2

```
lib_type_version.instantiate(device_name = "PLC_1", software_unit_name: "Unit1", folder_path: "group1/group2")
```

2.22.12. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = lib_type_version.get_supported_export_format()
```

2.22.13. **export(target_directory_path: str, export_format: str, library_export_options: Enums.LibraryExportOptions, keep_folder_structure: Optional[bool] = None)**

Export library type

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_format	str	Optional: Export format, by default SimaticSD, use <code>get_supported_export_format()</code> to get different formats
library_export_options	Enums.LibraryExportOptions	Optional: Export options of the Library by default <code>WithLibraryVersionInfoFile</code>
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
lib_type_version.export(library_export_options =
ts.Enums.LibraryExportOptions.WithLibraryVersionInfoFile, export_format = "SimaticML",
keep_folder_structure = true)
```

2.22.14. **edit(device_name: Optional[str] = None)**

Edit the library type version for a specific device.

Returns → `LibraryTypeVersion` The edited library type version object.

Parameters	Type	Description
device_name	str	Optional: Name of the device where to search instance(e.g., PLC or HMI)

```
edited_version = lib_type_version.edit(device_name="PLC_1")
```

2.22.15. **release(dependencies_mode: Enums.DependenciesMode, version: str, author: str, comment: str)**

Release the library type version.

Parameters	Type	Description
dependencies_mode	Enums.DependenciesMode	Mode for dependencies
version	str	Version string
author	str	Author name
comment	str	Release comment

```
lib_type_version.release(dependencies_mode=ts.Enums.DependenciesMode.AutomaticallyCreateOrReleaseDependenciesIfRequired, version="1.0.0", author="John", comment="Initial release")
```

2.22.16. **discard()**

Discard changes to the library type version.

```
lib_type_version.discard()
```

2.22.17. **set_as_default()**

Set this library type version as the default.

```
lib_type_version.set_as_default()
```

2.22.18. **get_comment()**

Get the comment for this library type version.

Returns → **str** The comment.

```
comment = lib_type_version.get_comment()
```

2.22.19. **get_identifier()**

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.23. Library - ProjectLibrary

The class represents a project library in the context of TIA Portal.

2.23.1. get_types()

Get the library types of the folder

Returns → `List[LibraryType]` List of types

```
types = projectlib.get_types()
```

2.23.2. find_library_type(library_type_name: str)

Find the library type by name

Returns → `LibraryType` Library type

Parameters	Type	Description
library_type_name	str	Name of the library type

```
type = projectlib.find_library_type(library_type_name = "MyType")
```

2.23.3. clean_up(clean_up_mode: Enums.CleanUpMode, folder_path: Optional[str] = None)

Cleans up the library to delete unused versions of types based on the cleanUpMode specified

Parameters	Type	Description
clean_up_mode	Enums.CleanUpMode	Clean up mode
folder_path	str	Optional: The target base path within the project library where the imported folder hierarchy will be recreated

```
projectlib.clean_up(clean_up_mode = siemens_tia_scripting.Enums.CleanUpMode.DeleteUnusedTypes)
```

2.23.4. import_library_types(import_root_directory: str, device_name: Optional[str] = None, software_unit_name: Optional[str] = None, plc_folder_path: Optional[str] = None, library_folder_path: Optional[str] = None)

Import library types a directory to the Project Library

Parameters	Type	Description
import_root_directory	str	The source directory path from which all files and folders will be imported
device_name	str	Optional: Name of the PLC Software
software_unit_name	str	Optional: Name of the PLC Software Unit from the PLC
plc_folder_path	str	Optional: The target base path within the PLC's group structure where the imported folder hierarchy will be recreated
library_folder_path	str	Optional: The target base path within the project library where the imported folder hierarchy will be recreated

```
projectlib.import_library_types(import_root_directory= "C:\\ws\\importLibraryTypes",
device_name: "PLC_1", software_unit_name: "Unit1", plc_folder_path: "group1/group2",
library_folder_path: "ImportLibFolder/Types")
```

2.23.5. **update_library(library_name: str, update_mode: int, delete_mode: int, conflict_mode: int, type_guids: Optional[List[str]] = None)**

Update target library with type(s) from source

Parameters	Type	Description
library_name	str	The name of the target Global library to update
update_mode	integer (enum)	[1: 'ForceSetAnyUpdatedVersionAsDefault', 2: 'NoDefaultVersionChange', 3: 'SetOnlyHigherUpdatedVersionAsDefault']
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
conflict_mode	integer (enum)	[1: 'CancelIfStructureConflicts', 2: 'RetainStructure', 3: 'UpdateStructure']
type_guids	List[str]	Optional: This option should be used to select specific source type or types. If empty, all types in project library are selected

```
project_lib.update_library(library_name = "MyGlobalLibrary", update_mode = 1, delete_mode = 1, conflict_mode = 3, type_guids = ["93826aed-7eac-4380-b4f4-f58e2707c862", "1fb57e17-f627-45f7-b7b8-644954b8888b"])
```

2.23.6. **update_project(delete_mode: int, type_guids: Optional[List[str]] = None)**

Update type instances in project using project library. If type_guids provided, only the selected types will be used for update

Parameters	Type	Description
delete_mode	integer (enum)	[0: 'AutomaticallyDelete', 1: 'DoNotDelete']
type_guids	List[str]	Optional: This option should be used to select specific source type or types. If empty, all types in project library are selected

```
project_lib.update_library(delete_mode: 1, type_guids: ["779c66b7-8fe2-46c0-a8a9-7da387b2521b", "bb856213-036d-47e4-9e8b-129e3c1606f6"])
```

2.23.7. **harmonize_project(harmonize_options: int, type_guids: Optional[List[str]] = None)**

Harmonize type instances in project using project library. If type_guids provided, only the selected types will be used for update

Parameters	Type	Description
harmonize_options	integer (enum)	[0: 'HarmonizePathsAndNames', 1: 'HarmonizePaths', 2: 'HarmonizeNames']
type_guids	List[str]	Optional: This option should be used to select specific source type or types. If empty, all types in project library are selected

```
project_lib.harmonize_project(harmonize_options: 0, type_guids: ["779c66b7-8fe2-46c0-a8a9-7da387b2521b", "bb856213-036d-47e4-9e8b-129e3c1606f6"])
```


2.23.8. **create_folder(folder_path: str)**

Create new folder at specified path in the library

Parameters	Type	Description
folder_path	str	Optional: The target base path within the project library where the imported folder hierarchy will be recreated

```
projectlib.create_folder(folder_path = "myFolder/subFolderExample")
```

2.23.9. **delete_folder(folder_path: str)**

Delete specified folder in project library

Parameters	Type	Description
folder_path	str	Optional: The target base path within the project library where the imported folder hierarchy will be recreated

```
projectlib.delete_folder(folder_path = "myFolder/subFolderExample")
```

2.24. PLC Data - ExternalSource

The class represents an external source in TIA Portal. It provides methods for getting information about the external source, retrieving properties and generating blocks.

2.24.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.24.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.24.3. block_gen()

Generate the blocks from the external source

```
ext_source.block_gen()
```

2.24.4. delete()

Delete the object

```
tiap_object.delete()
```

2.24.5. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.24.6. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object

Parameters	Type	Description
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.24.7. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.24.8. **get_identifier()**

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.25. PLC Data - ForceTable

The class represents a force table in TIA Portal. It provides methods for getting information about the force table, retrieving properties and exporting the table.

2.25.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.25.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.25.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.25.4. is_consistent()

Check if the force table is consistent

Returns → `bool` True if consistent

```
value = force_table.is_consistent()
```

2.25.5. show_in_editor()

Open the object in the editor

```
plc_object.show_in_editor()
```

2.25.6. **get_properties()**

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.25.7. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.25.8. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.25.9. **get_path_full()**

Get group path for an object until project

Returns → `str` group full path

```
group_path = plcdata.get_path_full()
```

2.25.10. **get_path()**

Get group path for an object until parent system folder

Returns → `str` group path

```
group_path = plcdata.get_path_full()
```

2.25.11. **get_fingerprints()**

Get all fingerprint data for the object

Returns → `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.25.12. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.26. PLC Data - NamedValueType

The class represents a Named value data type (NVT) in TIA Portal. It provides methods for managing and controlling Named value data types, exporting the data type, retrieving data type information and checking cross-references.

2.26.1. get_name()

Get the name of the Named value data type

Returns → `str` Name of the NVT

```
name = nvt.get_name()
```

2.26.2. get_namespace()

Get the name of the Named value data type namespace

Returns → `str` Name of the namespace

```
name = nvt.get_namespace()
```

2.26.3. export(target_directory_path: str, keep_folder_structure: Optional[bool] = None)

Export the Named value data type

If `keep_folder_structure` is set to true, all group names are represented hierarchically with folders.

Parameters	Type	Description
<code>target_directory_path</code>	<code>str</code>	Folder path for the export
<code>keep_folder_structure</code>	<code>bool</code>	True: All group names are represented hierarchically with folders

```
nvt.export(target_directory_path = "C:\\ws\\export")
```

2.27. PLC Data - PlcTag

The class provides methods to interact with TIA Portal PLC Tags. It allows you to get the name, properties and perform export operations on PLC Tags.

2.27.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.27.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.27.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.27.4. delete()

Delete the object

```
tiap_object.delete()
```

2.27.5. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```


2.27.6. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.27.7. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.27.8. **get_identifier()**

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.28. PLC Data - PlcTagTable

The represents a table of PLC tags in TIA Portal. It provides methods for managing and controlling PLC tags, exporting the table, retrieving tags and user constants and checking cross-references.

2.28.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.28.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.28.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.28.4. get_plc_tags()

Get the list of PLC tags

Returns → `List[PlcTag]` List of the PLC tags

```
plctags = table.get_plc_tags()
```

2.28.5. get_user_constants()

Get the list of the user constants

Returns → `List[UserConstant]` List of the PLC constants

```
constants = table.get_user_constants()
```

2.28.6. export_cross_references(target_directory_path: str, filter: int)

Export cross references of the PLC tag table filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameters	Type	Description
target_directory_path	str	Folder path for the export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
table.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences", filter = 2)
```

2.28.7. show_in_editor()

Open the object in the editor

```
plc_object.show_in_editor()
```

2.28.8. delete()

Delete the object

```
tiap_object.delete()
```

2.28.9. get_properties()

Get all properties of the object

Returns → List[str] Names of the attributes

```
properties = tiap_object.get_properties()
```

2.28.10. set_property(name:str, value: str)

Set the property of the object

Returns → int Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.28.11. get_supported_export_format()

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.28.12. `get_path_full()`

Get group path for an object until project

Returns → `str` group full path

```
group_path = plcdata.get_path_full()
```

2.28.13. `get_path()`

Get group path for an object until parent system folder

Returns → `str` group path

```
group_path = plcdata.get_path_full()
```

2.28.14. `get_fingerprints()`

Get all fingerprint data for the object

Returns → `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.28.15. `get_identifier()`

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.29. PLC Data - ProgramBlock

This represents a program block in TIA Portal. It provides various methods for managing and controlling program blocks.

2.29.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.29.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.29.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.29.4. compile()

Compile the program block

```
programblock.compile()
```

2.29.5. is_consistent()

Check if the program block is consistent

Returns → `bool` True if consistent

```
value = programblock.is_consistent()
```

2.29.6. export_cross_references(target_directory_path: str, filter: int)

Export cross references of the program block filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameters	Type	Description
target_directory_path	str	Folder path for the export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
programblock.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences",
filter = 2)
```

2.29.7. show_in_editor()

Open the object in the editor

```
plc_object.show_in_editor()
```

2.29.8. get_type_version_guid()

Get the GUID of the type version

Returns → str GUID

```
guid = plc_object.get_type_version_guid()
```

2.29.9. get_type_guid()

Get the GUID of the type

Returns → str GUID

```
guid = plc_object.get_type_guid()
```

2.29.10. delete()

Delete the object

```
tiap_object.delete()
```

2.29.11. get_properties()

Get all properties of the object

Returns → List[str] Names of the attributes

```
properties = tiap_object.get_properties()
```

2.29.12. get_path_full()

Get group path for an object until project

Returns → str group full path

```
group_path = plcdata.get_path_full()
```

2.29.13. get_path()

Get group path for an object until parent system folder

Returns → `str` group path

```
group_path = plcdata.get_path_full()
```

2.29.14. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.29.15. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.29.16. **get_fingerprints()**

Get all fingerprint data for the object

Returns → `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.29.17. **is_library_type()**

Check if the object is a library type

Returns → `bool` True if is a library type

```
value = plcdata.is_library_type()
```

2.29.18. **set_know_how_protection(password: str)**

Set know-how protection on the program block with the given password.

Parameters	Type	Description
password	str	Password for KnowHow Protection

```
programblock.set_know_how_protection(password = "Password!123")
```

2.29.19. **remove_know_how_protection(password: str)**

Remove know-how protection from the program block with the given password.

Parameters	Type	Description
password	str	Password for KnowHow Protection

```
programblock.remove_know_how_protection(password = "Password!123")
```

2.29.20. **get_identifier()**

Get the identifier of the object

Returns → **str** Identifier as string

```
property_value = tiap_object.get_identifier()
```


2.30. PLC Data - SafetyAdministration

The class provides methods to interact with TIA Portal's Safety Administration. It allows you to check the logged-on status, password status and perform import and export operations.

2.30.1. is_logged_on()

Check if user is logged in to Safety administration offline program

Returns → `bool` Status of login

```
status = safety_admin.is_logged_on()
```

2.30.2. is_password_set()

Check if the safety administration has a password set for the safety offline program

Returns → `bool` Status if password is set

```
status = safety_admin.is_password_set()
```

2.30.3. get_offline_serial_number()

Get the offline serial number

Returns → `str` Offline serial number

```
value = safety_admin.get_offline_serial_number()
```

2.30.4. export_config(target_directory_path: str)

Export the safety administration configuration

The folder "SafetyAdministration" will be automatically created on the `target_directory_path`, if not already exists.

Parameters	Type	Description
<code>target_directory_path</code>	<code>str</code>	Folder path for the export

```
safety_admin.export_config(target_directory_path = "C:\\ws\\export")
```

2.30.5. import_config(import_root_directory: str)

Import the safety administration configuration

Parameters	Type	Description
<code>import_root_directory</code>	<code>str</code>	Folder path for the import

```
safety_admin.import_config(import_root_directory = "C:\\ws\\exported\\SafetyAdministration")
```

2.31. PLC Data - SoftwareUnit

The class represents a software unit in TIA Portal. It provides methods for getting information about the software unit, exporting the software unit and accessing various subcomponents of the software unit, such as PLC tag tables, program blocks, system blocks, user data types and external sources.

2.31.1. get_name()

Get the name of the software unit

Return → `str` Name of the software unit *

```
name = software_unit.get_name()
```

2.31.2. compile()

Compile the software unit

```
software_unit.compile()
```

2.31.3. export_configuration(export_file_path: str)

Export the software unit configuration

Parameters	Type	Description
export_file_path	str	Full path for the configuration

```
software_unit.export_configuration(export_file_path = "C:\\ws\\export\\config.xml")
```

2.31.4. import_configuration(import_file_path: str)

Import the software unit configuration

Parameters	Type	Description
import_file_path	str	Full path for the configuration

```
software_unit.import_configuration(import_file_path = "C:\\ws\\export\\config.xml")
```

2.31.5. get_plc_tag_tables()

Get the list of the PLC tag tables

Returns → `List[PlcTagTable]` List of the PLC tag tables of the software unit

```
plc_tag_tables = software_unit.get_plc_tag_tables()
```

2.31.6. get_program_blocks()

Get the list of the program blocks

Returns → `List[ProgramBlock]` List of the program blocks of the software unit

```
blocks = software_unit.get_program_blocks()
```

2.31.7. get_system_blocks()

Get the list of the system blocks

Returns → `List[SystemBlock]` List of the system blocks of the software unit

```
blocks = software_unit.get_system_blocks()
```

2.31.8. **get_user_data_types()**

Get the list of the user data types

Returns → `List[UserDataTypes]` List of the user data types of the software unit

```
udts = software_unit.get_user_data_types()
```

2.31.9. **get_external_sources()**

Get the list of the external sources

Returns → `List[ExternalSource]` List of the external sources of the software unit

```
ext_sources = software_unit.get_external_sources()
```

2.31.10. **get_named_value_types()**

Get the list of the named value types

Returns → `List[NamedValueType]` List of the named value types of the software unit

```
nvts = software_unit.get_named_value_types()
```

2.31.11. **import_blocks(import_root_directory: str)**

Import program blocks from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
software_unit.import_blocks(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\Program blocks")
```

2.31.12. **import_plc_tags(import_root_directory: str)**

Import tags from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
software_unit.import_plc_tags(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC tags")
```

2.31.13. **import_data_types(import_root_directory: str)**

Import user data types from a directory to the PLC

Parameters	Type	Description
import_root_directory	str	Directory of the import folder

```
software_unit.import_data_types(import_root_directory = "C:\\ws\\importfolder\\PLC_1\\PLC data types")
```

2.31.14. **export_cross_references(target_directory_path: str, filter: int)**

Export cross references of the software unit object filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameters	Type	Description
target_directory_path	str	Folder path for the export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameters	Type	Description
		'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
software_unit.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences",
filter = 2)
```

2.31.15. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.31.16. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.31.17. delete()

Delete the object

```
tiap_object.delete()
```

2.31.18. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.31.19. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.32. PLC Data - SystemBlock

The class represents a system block in TIA Portal. It provides various methods for managing and controlling system blocks.

2.32.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.32.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.32.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.32.4. compile()

Compile the system block

```
system_block.compile()
```

2.32.5. is_consistent()

Check if the system block is consistent

Returns → `bool` True if consistent

```
value = system_block.is_consistent()
```

2.32.6. export_cross_references(target_directory_path: str, filter: int)

Export cross references of the system block filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameters	Type	Description
target_directory_path	str	Folder path for the export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
system_block.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences",
filter = 2)
```

2.32.7. show_in_editor()

Open the object in the editor

```
plc_object.show_in_editor()
```

2.32.8. delete()

Delete the object

```
tiap_object.delete()
```

2.32.9. get_properties()

Get all properties of the object

Returns → List[str] Names of the attributes

```
properties = tiap_object.get_properties()
```

2.32.10. get_path_full()

Get group path for an object until project

Returns → str group full path

```
group_path = plcdata.get_path_full()
```

2.32.11. get_path()

Get group path for an object until parent system folder

Returns → str group path

```
group_path = plcdata.get_path_full()
```

2.32.12. set_property(name:str, value: str)

Set the property of the object

Returns → int Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")

- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.32.13. get_supported_export_format()

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.32.14. get_fingerprints()

Get all fingerprint data for the object

Returns → `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.32.15. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.33. PLC Data - TechnologyObject

The class represents a technology object in TIA Portal. It provides various methods for managing and controlling technology objects.

2.33.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.33.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.33.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.33.4. compile()

Compile the technology object

```
to.compile()
```

2.33.5. is_consistent()

Check if the technology object is consistent

Returns → `bool` True if consistent

```
value = to.is_consistent()
```


2.33.6. delete()

Delete the object

```
tiap_object.delete()
```

2.33.7. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.33.8. get_path_full()

Get group path for an object until project

Returns → `str` group full path

```
group_path = plcdata.get_path_full()
```

2.33.9. get_path()

Get group path for an object until parent system folder

Returns → `str` group path

```
group_path = plcdata.get_path_full()
```

2.33.10. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.33.11. get_supported_export_format()

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.33.12. get_fingerprints()

Get all fingerprint data for the object

Returns -> `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.33.13. `get_identifier()`

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.34. PLC Data - UserConstant

The class represents a user constant in TIA Portal. It provides methods for getting information about the user constant, retrieving properties and exporting the constant.

2.34.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.34.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.34.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.34.4. delete()

Delete the object

```
tiap_object.delete()
```

2.34.5. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.34.6. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.34.7. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.34.8. **get_identifier()**

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.35. PLC Data - UserData Type

The class represents a PLC data type in TIA Portal. It provides methods for managing and controlling PLC data types, exporting the data type, retrieving data type information and checking cross-references.

2.35.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.35.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.35.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.35.4. compile()

Compile the PLC data type

```
udt.compile()
```

2.35.5. is_consistent()

Check if the PLC data type is consistent

Returns → `bool` True if consistent

```
value = udt.is_consistent()
```

2.35.6. export_cross_references(target_directory_path: str, filter: int)

Export the cross references of the PLC data type filter → integer - [1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

Parameters	Type	Description
target_directory_path	str	Folder path for the export
filter	integer (enum)	[1: 'AllObjects', 2: 'ObjectsWithReferences', 3: 'ObjectsWithoutReferences', 4: 'UnusedObjects']

```
udt.export_cross_references(target_directory_path = "C:\\ws\\exportcrossreferences", filter = 2)
```

2.35.7. get_type_version_guid()

Get the GUID of the type version

Returns → str GUID

```
guid = plc_object.get_type_version_guid()
```

2.35.8. get_type_guid()

Get the GUID of the type

Returns → str GUID

```
guid = plc_object.get_type_guid()
```

2.35.9. delete()

Delete the object

```
tiap_object.delete()
```

2.35.10. get_properties()

Get all properties of the object

Returns → List[str] Names of the attributes

```
properties = tiap_object.get_properties()
```

2.35.11. get_path_full()

Get group path for an object until project

Returns → str group full path

```
group_path = plcdata.get_path_full()
```

2.35.12. get_path()

Get group path for an object until parent system folder

Returns → str group path

```
group_path = plcdata.get_path_full()
```

2.35.13. **set_property(name:str, value: str)**

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.35.14. **get_supported_export_format()**

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.35.15. **get_fingerprints()**

Get all fingerprint data for the object

Returns → `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.35.16. **is_library_type()**

Check if the object is a library type

Returns → `bool` True if is a library type

```
value = plcdata.is_library_type()
```

2.35.17. **get_identifier()**

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.36. PLC Data - WatchTable

The class represents a watch table in TIA Portal. It provides methods for getting information about the watch table, retrieving properties and exporting the table.

2.36.1. get_name()

Get the name of the object

Returns → `str` Name of the object

```
name = tiap_object.get_name()
```

2.36.2. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.36.3. export(target_directory_path: str, export_options: Optional[Enums.ExportOptions] = None, export_format: Optional[Enums.ExportFormats] = None, keep_folder_structure: Optional[bool] = None)

Export the object

If `keep_folder_structure` is set to True, all group names are represented hierarchically with folders.

Parameters	Type	Description
target_directory_path	str	Folder path for the export
export_options	Enums.ExportOptions	Optional: Export options, by default ExportOptions.None
export_format	Enums.ExportFormats	Optional: Export format, by default SimaticML
keep_folder_structure	bool	True: All group names are represented hierarchically with folders

```
tiap_object.export(target_directory_path = "C:\\ws\\export", export_options = Enums.ExportOptions.WithDefaults)
```

2.36.4. is_consistent()

Check if the watch table is consistent

Returns → `bool` True if consistent

```
value = watch_table.is_consistent()
```

2.36.5. show_in_editor()

Open the object in the editor

```
plc_object.show_in_editor()
```


2.36.6. delete()

Delete the object

```
tiap_object.delete()
```

2.36.7. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.36.8. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.36.9. get_supported_export_format()

Returns list of supported export formats

Returns → `List[str]` supported format

```
supported_formats = plc_object.get_supported_export_format()
```

2.36.10. get_path_full()

Get group path for an object until project

Returns → `str` group full path

```
group_path = plcdata.get_path_full()
```

2.36.11. get_path()

Get group path for an object until parent system folder

Returns → `str` group path

```
group_path = plcdata.get_path_full()
```

2.36.12. get_fingerprints()

Get all fingerprint data for the object

Returns -> `List[Tuple[str, str]]` fingerprints

```
fingerprints = plcdata.get_fingerprints()
```

2.36.13. `get_identifier()`

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

2.37. Project - Project

The class represents a TIA Portal project and offers functions for common project operations, such as saving, closing, enabling/disabling simulation support, archiving, managing hardware and more.

2.37.1. get_portal()

Get the TIA Portal instance from the current project

```
portal = project.get_portal()
```

2.37.2. save()

Save the TIA Portal project

```
project.save()
```

2.37.3. close()

Closes the TIA Portal project without saving any pending changes

WARNING: All unsaved modifications will be permanently lost

```
project.close()
```

2.37.4. set_simulation_support(value: bool)

Set the simulation support for the TIA Portal project

Parameters	Type	Description
value	bool	Set the value of the simulation support

```
project.set_simulation_support(value = True)
```

2.37.5. archive(target_directory_path: str, archive_name: str, delete_existing_archive: bool)

Archive the TIA Portal project

Returns → **str** Full path of the new archive

Parameters	Type	Description
target_directory_path	str	Folder path where archive should be placed
archive_name	str	name of the archive file
delete_existing_archive	bool	Set if the output file should be deleted first

```
archive_fullpath = project.archive(target_directory_path = "C:\\ws\\newfolder", archive_name = "testproj", delete_existing_archive = True)
```

2.37.6. save_as(target_directory_path: str, project_name: str)

Save the TIA Portal project under another path and name

Returns → **str** Full path of the saved project

Parameters	Type	Description
target_directory_path	str	Folder path where project should be saved

Parameters	Type	Description
project_name	str	Name of the project name

```
project_file_path = project.save_as(target_directory_path = "C:\\ws\\newfolder", project_name = "testproj")
```

2.37.7. export_cax_data(export_file_path: str, log_file_path: str)

Export the CAX data of the TIA Portal project

Returns → `bool` Result state of the export

Parameters	Type	Description
export_file_path	str	Full path of the export file
log_file_path	str	Full path of the log file

```
result = project.export_cax_data(export_file_path = "C:\\ws\\exportfolder\\exportCAX.aml", log_file_path = "C:\\ws\\exportfolder\\exportCAX.log")
```

2.37.8. import_cax_data(import_file_path: str, log_file_path: str)

Import CAX data to the TIA Portal project

Returns → `bool` Result state of the import

Parameters	Type	Description
import_file_path	str	Fullpath of the import file
log_file_path	str	Fullpath of the log file

```
result = project.import_cax_data(import_file_path = "C:\\ws\\importfolder\\importCAX.aml", log_file_path = "C:\\ws\\importfolder\\importCAX.log")
```

2.37.9. open_topology_editor()

Open the topology editor in TIA Portal for the TIA Portal project

```
project.open_topology_editor()
```

2.37.10. open_network_editor()

Open the network editor in TIA Portal for the TIA Portal project

```
project.open_network_editor()
```

2.37.11. get_plcs()

Get a list of PLCs in the TIA Portal project

Returns → `List[Plc]` All PLC's as list

```
plcs = project.get_plcs()
for plc in plcs:
    ...
```

2.37.12. get_hmis()

Get a list of HMIs in the TIA Portal project

Returns → `List[Hmi]` All HMI's as list

```
hmis = project.get_hmis()
for hmi in hmis:
...
```

2.37.13. get_application_tests()

Get a list of Test Suite application tests in the TIA Portal project

Needs Test Suite installed and licensed.

Returns → `List[ApplicationTest]` All Application tests as list

```
tests = project.get_application_tests()
```

2.37.14. get_system_tests()

Get a list of Test Suite system tests in the TIA Portal project

Needs Test Suite installed and licensed.

Returns → `List[SystemTest]` All System tests as list

```
tests = project.get_system_tests()
```

2.37.15. get_rule_sets()

Get a list of Test Suite Styleguide rule sets in the TIA Portal project

Needs Test Suite installed and licensed.

Returns → `List[RuleSet]` All Rule sets as list

```
rule_sets = project.get_rule_sets()
```

2.37.16. get_project_library()

Get the project library

Returns → `ProjectLibrary` Project library

```
project_lib = project.get_project_library()
```

2.37.17. web_block_generate()

Generate Web block DBs according the user define pages

Web server must activated.

```
project.web_block_generate()
```

2.37.18. upgrade_hardware(full_upgrade: bool)

Upgrade the devices to the latest order number and firmware version

If `full_upgrade` is set to True, all devices will be changed to the newest available order number (Device type) and firmware:

from CPU1511F 6ES7 511-1FK00-0AB0 V1.7 to 6ES7 511-1FK00-0AB0 V1.8.

With full upgrade: from CPU1511F 6ES7 511-1FK00-0AB0 V1.7 to 6ES7 511-1FL03-0AB0 V3.0.

Parameters	Type	Description
full_upgrade	bool	Set if for full upgrade (latest order number and firmware version)

```
project.upgrade_hardware(full_upgrade = True)
```

2.37.19. **sivarc_generate()**

Start SiVArc-generation

SiVArc must be installed and licensed.

```
project.sivarc_generate()
```

2.37.20. **update_module_description()**

Update the module description of all devices in the project

```
project.update_module_description()
```

2.37.21. **set_virtual_plc_support(value: bool)**

Set virtual PLC support in the project

Available only in V19 or higher

Parameters	Type	Description
value	bool	Set the value of the virtual support

```
project.set_virtual_plc_support(value = "True")
```

2.37.22. **import_umac_config(import_file_path: str)**

Import UMAC and UMC Configuration to the TIA Portal project

Returns → **int** Result state ReturnCode of the import

Parameters	Type	Description
import_file_path	str	Full path of the import file
secret	str	Secret for the encryption (optional if passwords encrypted)
secret_env_name	str	Name of the environment variable where Secret is stored (optional if passwords encrypted)

```
project.import_umac_config(import_file_path = "C:\\ws\\importfolder\\importUMAC.json",
secret_env_name = "MYSECRETEENV")
```

2.37.23. **export_umac_config(export_file_path: str)**

Export UMAC and UMC configuration from the TIA Portal project

If export file already exists, it will be overwritten

Parameters	Type	Description
export_file_path	str	Full path of the export file

```
project.export_umac_config(export_file_path = "C:\\ws\\exportfolder\\exportUMAC.json")
```

2.37.24. **import_password_policy(import_file_path: str)**

Import Password Policies to the TIA Portal project

Parameters	Type	Description
import_file_path	str	Full path of the import file

```
project.import_password_policy(import_file_path = "C:\\ws\\importfolder\\PWPolicy.json")
```

2.37.25. export_password_policy(export_file_path: str)

Export Password Policies from the TIA Portal project

If export file already exists, it will be overwritten

Parameters	Type	Description
export_file_path	str	Full path of the import file

```
project.export_password_policy(export_file_path = "C:\\ws\\exportfolder\\exportPWPolicy.json")
```

2.37.26. delete()

Delete the TIA Portal project

```
project.delete()
```

2.37.27. start_transaction(undo_text: str, dialog_text: str)

Start exclusive access and new transaction

Parameters	Type	Description
undo_text	str	Text on the Undo button
dialog_text	str	Text on the dialog during the transaction

```
project.start_transaction(undo_text = "MyUndoDescription", dialog_text = "MyExclusiveAccess")
```

2.37.28. end_transaction(rollback: Optional[bool] = None)

End transaction and exclusive access

Parameters	Type	Description
rollback	bool	Set if changes during transaction should be rolled back (default: false)

```
project.end_transaction()
```

2.37.29. update_transaction(dialog_text: str)

Update transaction of the running exclusive access

Parameters	Type	Description
dialog_text	str	Text on the dialog during the transaction

```
project.update_transaction(dialog_text = "MyNewExclusiveAccess")
```

2.37.30. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → **str** Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.37.31. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.37.32. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.37.33. is_session()

Checks if current project is a local session

Returns → `bool` True if local session

```
is_session = project.is_session()
```

2.37.34. commit_and_close(commit_message: str)

Commit message and close the project

Returns → `int` Revision number of the commit

Parameters	Type	Description
commit_message	str	Commit message

```
rev_number = project.commit_and_close(commit_message = "MyCommitMessage")
```

2.37.35. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```


2.38. Project - ProjectServer

The class represents a TIA Project-Server in TIA Portal.

2.38.1. get_host()

Get the host of the TIA Project-Server

Returns → `str` Host

```
host = server_project.get_host()
```

2.38.2. get_port()

Get the port of the TIA Project-Server

Returns → `int` Port number

```
port = server_project.get_port()
```

2.38.3. get_server_name()

Get the server name of the TIA Project-Server

Returns → `str` Server name

```
name = server_project.get_server_name()
```

2.38.4. print_info()

Show information of the TIA Project-Server

```
server_project.print_info()
```

2.38.5. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.38.6. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.38.7. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"

- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.38.8. delete()

Delete the object

```
tiap_object.delete()
```

2.38.9. add_project(project_path: str)

Add project to the project server

Parameters	Type	Description
project_path	str	Path of the projectfile

```
project_server.add_project(project_path = "D:\tia_sessions\MyTestSession\MyTestSession.als")
```

2.38.10. create_local_session(server_project_path: str, session_directory_path: str, session_name: str, exclusive: Optional[bool] = None)

Create a local session and return the project-path

Warning: only ungrouped server projects are supported and can be retrieved. Group servers are supported from API V20.

Returns → `str` TIA Portal project local session fullpath

Parameters	Type	Description
server_project_path	str	Path of the project on Project-Server
session_directory_path	str	Local path of the session directory
session_name	str	Name of the session
exclusive	bool	Default: true Opens exclusive session

```
local_session_project_path = project_server.create_local_session(server_project_path = "myServerGroup/MyServerProject", session_directory_path = "C:\\ws\\sessionDir", session_name = "myLocalSession", exclusive = False)
```

2.38.11. get_server_project_paths(group: Optional[str] = None)

Get all the project names from the selected TIA Project-Server

Returns → `List[str]` Paths of the projects

Parameters	Type	Description
group	str	Optional: group name

```
project_names = project_server.get_server_project_paths(group = "MyGroup")
```

2.38.12. delete_local_session(session_file_path: str, server_project_path: str)

Delete the local session

Parameters	Type	Description
session_file_path	str	Local session file path, supports .amc and also .als filepath
server_project_path	str	Path of the project on server

```
project_server.delete_local_session(session_file_path = "C:\\ws\\tia-  
sessions\\myproject\\myproject.amc" , server_project_path = "myGroup/serverproject")
```

2.39. Test Suite - ApplicationTest

The class represents an Test Suite application test in the context of TIA Portal.

Test Suite Advanced must be installed and licensed.

2.39.1. get_name()

Get the name of the Test Suite application test

Returns → `str` Name of the application test

```
name = app_test.get_name()
```

2.39.2. get_property(name: str)

Get the property of the Test Suite application test

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property of the Application Test

```
property_value = app_test.get_property(name = "Property")
```

2.39.3. export(target_directory_path: str)

Export the Test Suite application test

The folder "Application tests" will be automatically created on the `target_directory_path`, if not already exists.

Parameters	Type	Description
target_directory_path	str	Folder path for the export

```
app_test.export(target_directory_path = "C:\\ws\\export")
```

2.39.4. set_scope(plc_name: str, instance_name: Optional[str] = None, execution_mode: Optional[int] = None)

Set the scope of the Test Suite application test

Parameters	Type	Description
plc_name	str	Name of the PLC to be used
instance_name	str	PLCSIM Instance name (only V19 and higher)
execution_mode	integer (enum)	(only V19 and higher) [1: 'SystemManagedPLCSIMInstance', 2: 'ExternallyManagedPLCSIMInstance']

```
app_test.set_scope(plc_name = "PLC_1")
```

2.40. Test Suite - RuleSet

The class represents a Test Suite style guide rule set in the context of TIA Portal.

Test Suite Advanced must be installed and licensed.

2.40.1. get_name()

Get the name of the Test Suite style guide rule set

Returns → `str` Name of the rule set

```
name = rule.get_name()
```

2.40.2. get_property(name: str)

Get the property of the Test Suite style guide rule set

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
name	str	Property of the rule set

```
property_value = rule.get_property(name = "Property")
```

2.40.3. export(target_directory_path: str)

Export the Test Suite style guide rule set

The folder "Style guide" will be automatically created on the `target_directory_path`, if not already exists.

Parameters	Type	Description
target_directory_path	str	Folder path for the export

```
rule.export(target_directory_path = "C:\\ws\\export")
```

2.41. Test Suite - SystemTest

The class represents a Test Suite system test in the context of TIA Portal.

Test Suite Advanced must be installed and licensed.

2.41.1. get_name()

Get the name of the Test Suite system test

Returns → `str` Name of the system test

```
name = sys_test.get_name()
```

2.41.2. export(target_directory_path: str)

Export the Test Suite system test

The folder "System tests" will be automatically created on the `target_directory_path`, if not already exists.

Parameters	Type	Description
<code>target_directory_path</code>	<code>str</code>	Folder path for the export

```
sys_test.export(target_directory_path = "C:\\ws\\export")
```

2.41.3. set_scope(opcua_server_address: str, opcua_server_interface_type: int, opcua_server_interface_folder_path: Optional[str] = None)

Set the scope of the Test Suite system test

Parameters	Type	Description
<code>opcua_server_address</code>	<code>str</code>	OPC UA server address e.g. <code>opc.tcp://server.port/path</code>
<code>opcua_server_interface_type</code>	integer (enum)	[1: 'UserDefined', 2: 'StandardSIMATIC']
<code>opcua_server_interface_folder_path</code>	<code>str</code>	Folder path to interface files

```
sys_test.set_scope(opcua_server_address = "opc.tcp://server.port/path",  
opcua_server_interface_type = 1)
```

2.41.4. get_property(name: str)

Get the property of the object

Properties which are not string will be converted to string if possible.

Returns → `str` Value of the property as string

Parameters	Type	Description
<code>name</code>	<code>str</code>	Property name of the object

```
property_value = tiap_object.get_property(name = "CreationDate")
```

2.41.5. get_properties()

Get all properties of the object

Returns → `List[str]` Names of the attributes

```
properties = tiap_object.get_properties()
```

2.41.6. set_property(name:str, value: str)

Set the property of the object

Returns → `int` Return code of the result

Sets a property of the object by assigning the given value, provided it is in the correct format.

- **Boolean:** "True" or "False"
- **Integer:** Whole numbers (e.g., "42")
- **Double:** Decimal numbers with a dot (e.g., "3.14")
- **Enum:** Either the option index (e.g., "0", "1", "2") or the corresponding string value (e.g., "OptionA", "OptionB").

Parameters	Type	Description
name	str	Name of the property from the object
value	str	String value for the property e.g. "1", "True", "MyNewString"

```
is_set = tiap_object.set_property(name = "Name", value = "MyNewName")
```

2.41.7. get_identifier()

Get the identifier of the object

Returns → `str` Identifier as string

```
property_value = tiap_object.get_identifier()
```

3. Appendix

3.1. Service and support

SiePortal

The integrated platform for product selection, purchasing and support - and connection of Industry Mall and Online support. The SiePortal home page replaces the previous home pages of the Industry Mall and the Online Support Portal (SIOS) and combines them.

- **Products & Services**
In Products & Services, you can find all our offerings as previously available in Mall Catalog.
- **Support**
In Support, you can find all information helpful for resolving technical issues with our products.
- **mySieportal**
mySiePortal collects all your personal data and processes, from your account to current orders, service requests and more. You can only see the full range of functions here after you have logged in.

You can access SiePortal via this address: sieportal.siemens.com

Technical Support

The Technical Support of Siemens Industry provides you fast and competent support regarding all technical queries with numerous tailor-made offers – ranging from basic support to individual support contracts.

Please send queries to Technical Support via Web form: support.industry.siemens.com/cs/my/src

SITRAIN – Digital Industry Academy

We support you with our globally available training courses for industry with practical experience, innovative learning methods and a concept that's tailored to the customer's specific needs.

For more information on our offered trainings and courses, as well as their locations and dates, refer to our web page: siemens.com/sitrain

Industry Online Support app

You will receive optimum support wherever you are with the "Siemens Industry Online Support" app. The app is available for iOS and Android:



3.2. Links and literature

Table 4-1

No.	Topic
\1	SiePortal https://sieportal.siemens.com/
\2	Link to the entry page of this application example https://support.industry.siemens.com/cs/ww/en/view/109742322

3.3. Change documentation

Table 4-2

Version	Date	Modifications
V1.2.1	12/2025	fix: Hotfix TIA Portal Openness V21 Support
V1.2.0	12/2025	- feat: add TIA Portal Openness V21 Support - feat: add PLC Download and go online functionality
V1.1.0	09/2025	- feat!: move session handling to ProjectServer-object - feat: add functionality for HMI (Screens, Tags, Scripts, Connections, Cycles, Alarms, Text- and Graphic-lists,...) - feat: add support to export and import in the format SIMATIC SD (Source Documents) - feat: add support to set object-properties - feat: add support for library-use cases (handle folders, find types, instantiate, update, cleanup, edit, release, discard, export) - feat: add support for PLC objects (Know-How-Protection, Fingerprints) - feat: add functionality to create new projects and libraries - feat: add support for Project-Server groups - feat: add support for Object-Identifier - feat: update sample-scripts - fix: improvements from user-feedback
V1.0.7	10/2024	Initial release